

LT2: Linear-Time Looped Transformers

Chunyuan Deng¹, Yizhe Zhang², Rui-jie Zhu³, Yuanyuan Xu¹, Jiarui Liu⁴, T. S. Eugene Ng¹ and Hanjie Chen¹

¹Rice University, ²Apple, ³UC Santa Cruz, ⁴Carnegie Mellon University

Looped Transformers (LT) have emerged as a powerful architecture by iterating their layers multiple times before decoding the final token. However, their pairing with full attention retains quadratic complexity making it computationally expensive and slow. We introduce **LT2 (Linear-Time Looped Transformers)**, a family of looped architectures that replace quadratic softmax attention with subquadratic attention with linear-time complexity. We study two variants: *LT2-linear* with linear attention and *LT2-sparse* with sparse attention. We find looping uniquely synergizes with these variants: it enables iterative memory refinement in linear attention and progressively expands the effective receptive field in sparse attention. We formalize these benefits theoretically and demonstrate consistent empirical gains across controlled recall, state-tracking, and language modeling tasks. We then explore **LT2-hybrid**, a hybrid architecture that combines different attention variants in a looped setting. We find two architectural variants promising: (1) LT2-hybrid (GDN+DSA), which interleaves linear and sparse attention to *maximize efficiency*, matching the standard looped transformer’s quality at fully linear-time cost. and (2) LT2-hybrid (Full+GDN), which interleaves GDN with a small fraction of full attention layers to *maximize quality*, surpassing the standard looped transformer in both performance and efficiency. Furthermore, we also show how to turn a pre-trained LT into an LT2-hybrid model. With only about 1B tokens of training, our converted model (*Ouro-hybrid-1.4B*) outperforms industry-level 1B models and is competitive with industry-level 4B models while keeping the speed benefits of linear-time attention. Together, these two directions show a clear path to making looped transformers a more scalable architecture for language modeling and advancing the development of efficient, capable small language models.



Codebase: <https://github.com/chili-lab/LT2>



Huggingface Checkpoints: <https://huggingface.co/chili-lab/Ouro-hybrid-1.4B>

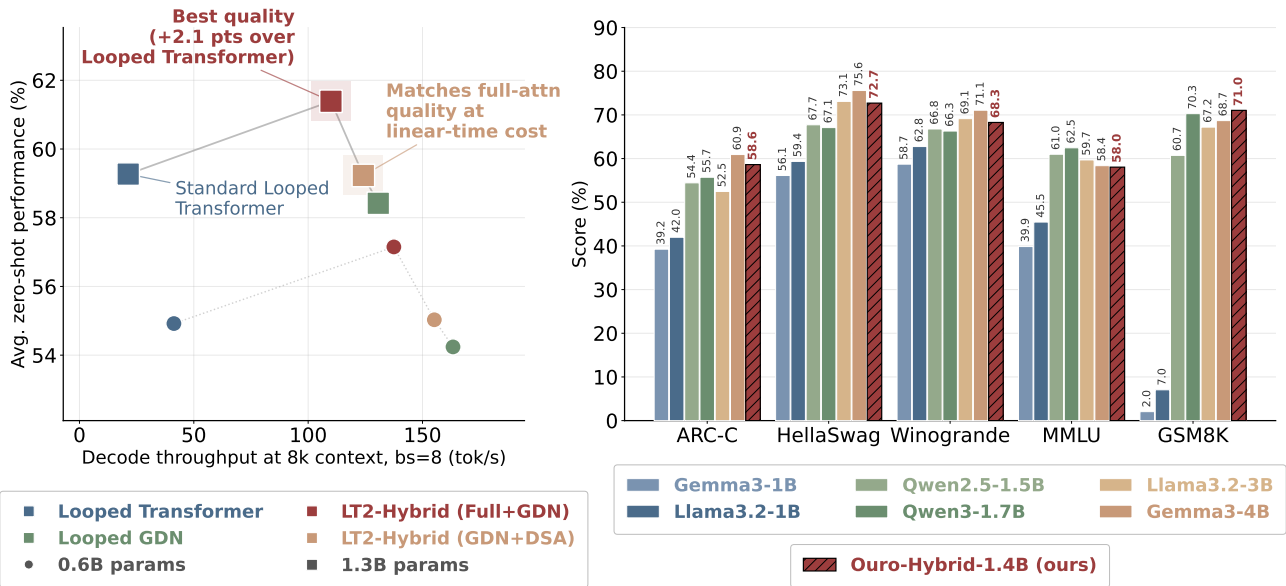


Figure 1: (Left) New parameter-efficiency frontier introduced by LT2. **(Right)** Converted LT2-Hybrid outperforms similarly sized industry-level 1B while matching 4B ones.

1. Introduction

Scaling neural language models along the parameter axis has driven much of modern NLP’s progress [7, 34, 29]. A complementary axis—scaling depth via weight-shared recurrence—has recently emerged as a promising alternative. These architectures, often called looped transformers (LT, originally *Universal Transformers* [16]), reuse the same weights across multiple steps before decoding the final prediction token [23, 69, 76]. In effect, repeated computation becomes effective depth: the model performs several rounds of latent computation while keeping the unique parameter count fixed, making looped transformers an appealing approach to parameter-efficient reasoning.

However, current looped transformers scale poorly because each loop has to re-apply quadratic full attention over the entire sequence repeatedly. Its cost and inference-time storage therefore grow with sequence length, and compound with each loop iteration. As a result, even though parameters are reused, training-time attention FLOPs and inference-time KV-cache usage scale poorly with the number of loops, making attention the dominant bottleneck in scaling looped transformers [62, 76]. As Figure 2 shows, processing every token through attention for T iterations causes both training-time attention FLOPs and inference-time KV-cache memory to grow substantially. At long contexts the quadratic attention term dominates, and adding loop steps quickly becomes impractical [76].

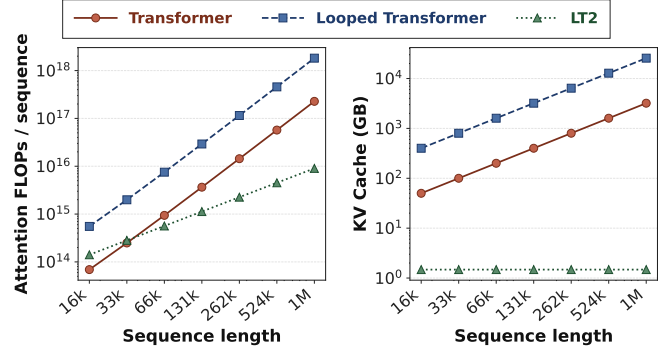


Figure 2: Attention FLOPs and inference cache memory vs. sequence length for a 1.3B model.

We introduce **LT2 (Linear-Time Looped Transformers)**, a family of looped architectures that replace quadratic softmax attention with subquadratic token-mixing primitives. We primarily study two distinct variants, LT2-linear and LT2-sparse, which replace the quadratic attention with linear attention [35, 72, 63] and sparse attention [68, 15], respectively. We show that looped operation can *turn compute into context*: it enables finer-grained control over recurrent memory in linear attention and enlarges the receptive field in sparse attention; we provide intuition in § 2.2 and a detailed theoretical analysis in Appendix B.1. Furthermore, we explore **LT2-hybrid**, a hybrid architecture that pushes the performance-efficiency frontier to a new level by mixing different attention variants in the looped setting. We demonstrate that LT2-hybrid (GDN [70] + DSA [15])—which combines linear and sparse attention within a looped setting—matches the standard looped transformer’s quality (59.3% avg. zero-shot) while delivering $\sim 5.7\times$ higher decode throughput at 8k context (125 vs. 22 tokens/s, batch size 8), entirely without quadratic attention. LT2-hybrid (Full + GDN), which interleaves GDN with a small fraction of full-attention layers, goes further: it improves average zero-shot performance by +2.1 points over the standard looped transformer (61.4% vs. 59.3%) while still achieving $\sim 5\times$ higher decode throughput at the same setting, and consistently outperforms the standard looped transformer across language modeling, recall, state-tracking, and efficiency benchmarks (§ 3).

Finally, we explore distilling a pretrained looped transformer (specifically, Ouro [76]) into an LT2 model. As shown in Figure 1 (right), with only ~ 1 B tokens of continued training, our converted *Ouro-Hybrid-1.4B* retains the quality of its full-attention teacher while inheriting LT2’s linear-time efficiency. The resulting model is competitive with industry-level open-source models in the 1B–4B parameter range across standard zero-shot benchmarks, matching or exceeding 1B-class baselines and approaching 3B–4B models on several tasks. This demonstrates that practitioners need not retrain from scratch: existing

Table 1: Token mixers supported by LT2. Blue highlights **gating/retention** and burnt sienna highlights **DPLR-style** operations. Train FLOPs are reported per layer for a sequence of length L ; cache/state memory is per layer at inference. w denotes the sparse-attention window/budget size with $w \ll L$.

Family	Mixer	State update rule	Train FLOPs	Cache / State mem.
Full attn.	Softmax MHA	$(\mathbf{K}_t, \mathbf{V}_t) = ([\mathbf{K}_{t-1}; \mathbf{k}_t], [\mathbf{V}_{t-1}; \mathbf{v}_t])$	$\mathcal{O}(L^2 d)$	$\mathcal{O}(Ld)$
Linear attn. (LT2-LA)	LA [35]	$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{k}_t \mathbf{v}_t^\top$	$\mathcal{O}(L d_k d_v)$	$\mathcal{O}(d_k d_v)$
	RetNet [59]	$\mathbf{S}_t = \gamma \mathbf{S}_{t-1} + \mathbf{k}_t \mathbf{v}_t^\top$	$\mathcal{O}(L d_k d_v)$	$\mathcal{O}(d_k d_v)$
	Mamba2 [14]	$\mathbf{S}_t = \alpha_t \mathbf{S}_{t-1} + \mathbf{k}_t \mathbf{v}_t^\top$	$\mathcal{O}(L d_k d_v)$	$\mathcal{O}(d_k d_v)$
	GLA [71]	$\mathbf{S}_t = \text{Diag}(\alpha_t) \mathbf{S}_{t-1} + \mathbf{k}_t \mathbf{v}_t^\top$	$\mathcal{O}(L d_k d_v)$	$\mathcal{O}(d_k d_v)$
	HGRN2 [50]	$\mathbf{S}_t = \text{Diag}(\alpha_t) \mathbf{S}_{t-1} + (\mathbf{I} - \alpha_t) \mathbf{v}_t^\top$	$\mathcal{O}(L d_k d_v)$	$\mathcal{O}(d_k d_v)$
	DeltaNet [54, 72]	$\mathbf{S}_t = (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top$	$\mathcal{O}(L d_k d_v)$	$\mathcal{O}(d_k d_v)$
	GDN [70]	$\mathbf{S}_t = \alpha_t (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top$	$\mathcal{O}(L d_k d_v)$	$\mathcal{O}(d_k d_v)$
	KDA [63]	$\mathbf{S}_t = (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \text{Diag}(\alpha_t) \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top$	$\mathcal{O}(L d_k d_v)$	$\mathcal{O}(d_k d_v)$
Sparse attn. (LT2-SA)	Window	$(\mathbf{K}_t, \mathbf{V}_t) = (\mathbf{K}_{[t-w:t]}, \mathbf{V}_{[t-w:t]})$ (sliding cache)	$\mathcal{O}(L w d)$	$\mathcal{O}(w d)$
	NSA [75]	KV cache + compressed blocks; $\mathcal{I}_t$: top- w selected indices	$\mathcal{O}(L w d)$	$\mathcal{O}(L d)$
	DSA [15]	KV cache; $\mathcal{I}_t$: top- w via lightning indexer	$\mathcal{O}(L w d)$	$\mathcal{O}(L d)$

looped transformers can be efficiently converted into linear-time variants, lowering the cost barrier to adopting the LT2 family models.

2. LT2: Linear-Time Looped Transformer

2.1. Architecture

Looped Transformer (LT). Let L denote sequence length and d the hidden dimension; we write the hidden-state sequence as $\mathbf{h} \in \mathbb{R}^{L \times d}$ and the state at position t as $\mathbf{h}_t \in \mathbb{R}^d$. A standard Transformer of depth N stacks N independently-parameterized blocks $\{\mathcal{F}_\ell\}_{\ell=1}^N$, each consisting of a token mixer and a position-wise FFN with residual connections:

$$\mathcal{F}_\ell(\mathbf{h}) = \mathbf{h}' + \text{FFN}_\ell(\mathbf{h}'), \quad \mathbf{h}' = \mathbf{h} + \text{MHA}_\ell(\mathbf{h}), \quad (1)$$

where MHA_ℓ is multi-head self-attention (we omit pre-norm for brevity). A *Looped Transformer* (LT) reuses these N shared blocks for T iterations:

$$\mathbf{h}^{(0)} = \text{Emb}(\mathbf{x}), \quad \mathbf{h}^{(\tau)} = (\mathcal{F}_N \circ \dots \circ \mathcal{F}_1)(\mathbf{h}^{(\tau-1)}), \quad \tau = 1, \dots, T, \quad \hat{\mathbf{y}} = \text{Dec}(\mathbf{h}^{(T)}), \quad (2)$$

yielding effective depth $T \cdot N$ with only N unique parameter sets—a $T \times$ parameter reduction over a Transformer of equivalent depth. Following Ouro [76], we use a fixed T throughout pre-training and we discuss adaptive computation time in the Appendix A.

LT2. LT2 simply replaces the MHA sub-layer in Eq. (1) with a subquadratic token mixer, so each shared block becomes

$$\mathcal{F}_\ell(\mathbf{h}) = \mathbf{h}' + \text{FFN}_\ell(\mathbf{h}'), \quad \mathbf{h}' = \mathbf{h} + \text{LinearMixer}_\ell(\mathbf{h}), \quad (3)$$

where LinearMixer_ℓ is any linear- or sparse-attention primitive in Table 1. Throughout, $\mathbf{q}_t, \mathbf{k}_t \in \mathbb{R}^{d_k}$ and $\mathbf{v}_t \in \mathbb{R}^{d_v}$ denote the query/key/value projections of $\mathbf{h}_t$; $\mathbf{S}_t \in \mathbb{R}^{d_k \times d_v}$ is the recurrent state of a linear-attention mixer. We additionally insert a zero-initialized, per-channel learned gate $\rho_\tau \in \mathbb{R}^d$ as a residual across loop iterations, $\mathbf{h}^{(\tau)} = \tilde{\mathbf{h}}^{(\tau)} + \rho_\tau \odot \mathbf{h}^{(\tau-1)}$, where $\tilde{\mathbf{h}}^{(\tau)}$ is the output of the looped block stack at iteration τ (i.e., $\tilde{\mathbf{h}}^{(\tau)} = (\mathcal{F}_N \circ \dots \circ \mathcal{F}_1)(\mathbf{h}^{(\tau-1)})$). Thus our setup includes two levels of residual connections: a traditional per-block identity residual connection and a learned per-loop residual.

2.2. Beyond Efficiency: Benefits of Looping

Subquadratic attention provides clear efficiency gains. A more interesting question is what looping adds to these attention variants. We make two claims: with T loop iterations, a diagonal-plus-low-rank (DPLR)

linear-attention block turns its rank-1 state update into a rank- T update, and a sliding-window block turns its window of size w into an effective receptive field of size Tw .

Loop $\times$ DPLR linear attention: rank- T update on recurrent memory. Frontier linear-attention architectures now use DPLR mixers, e.g. GDN [70], KDA [63], and RWKV7 [47]. We take KDA as our running example, which maintains a recurrent state $\mathbf{S}_t \in \mathbb{R}^{d_k \times d_v}$ at sequence position t via

$$\mathbf{S}_t = \mathbf{A}_t \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top, \quad \mathbf{A}_t = \text{Diag}(\alpha_t)(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top), \quad (4)$$

where $\alpha_t \in [0, 1]^{d_k}$ is a diagonal gate, so $\mathbf{A}_t$ is identity ($\mathbf{I}$) plus a rank-1 ($\mathbf{k}_t \mathbf{k}_t^\top$) perturbation. Prior work shows that a single such block can only model permutations of two elements per token, and cannot solve the S_n word problem ($n \geq 3$) in finite precision [27]. When the *same* shared block is looped T times, each loop iteration $\tau \in \{1, \dots, T\}$ contributes a fresh DPLR factor $\mathbf{A}_t^{(\tau)}$ acting on the recurrent state at position t , so the cumulative per-token state-transition operator becomes

$$\mathbf{A}_t^{\text{eff}} = \prod_{\tau=1}^T \mathbf{A}_t^{(\tau)} = \prod_{\tau=1}^T \text{Diag}(\alpha_t^{(\tau)}) (\mathbf{I} - \beta_t^{(\tau)} \mathbf{k}_t^{(\tau)} \mathbf{k}_t^{(\tau)\top}). \quad (5)$$

DeltaProduct [57] shows that the expressivity gains of looped DPLR depend on the relationships among the loop-specific keys $\{\mathbf{k}_t^{(\tau)}\}_{\tau=1}^T$. In one extreme, if all keys are identical, then each loop iteration erases the same direction in recurrent memory, yielding no expressivity gain over the non-looped case. In the other extreme, if the keys from different loop iterations are orthogonal, then the loop erases historical information along T distinct directions. In this case, the original transition, which contains a single rank-1 perturbation, is replaced by an effective transition with a rank- T memory-erasure subspace. We detailed discuss the proof in Appendix B.1.

Loop $\times$ sparse attention: receptive-field expansion. A single sliding-window block with window w lets each query at position t attend only to the last w tokens,

$$\mathcal{I}_t^{(1)} = \{t - w + 1, \dots, t\},$$

so the per-loop receptive field is $\mathcal{O}(w)$ and anything beyond the w tokens is invisible. Looping the block re-runs the same window over the sequence, and information moves further with every loop iteration: at loop iteration τ , position t attends to a window of loop- $(\tau-1)$ states, and those states have already absorbed information from their own windows at loop iteration $\tau-2$, and so on. Chaining this argument inductively (Appendix B.2) gives the receptive field after T loop iterations:

$$\mathcal{I}_t^{(T)} \supseteq \{\max(1, t - Tw + 1), \dots, t\}, \quad |\mathcal{I}_t^{(T)}| = \mathcal{O}(Tw). \quad (6)$$

In other words, T loops of a window- w block reach as far back as T stacked layers of window- w attention [9], but with $T \times$ fewer parameters. Looping therefore *turns compute into context*: once T is moderately large, a small fixed window already covers long sequences, which makes sparse mixers a natural partner for looping in long-context settings.

2.3. Hybrid LT2: Mixing Mixers Across Depth and Loops

We further explore hybrid architectures in the loop setting. A common practice in hybrid models is to interleave linear blocks with full-attention blocks to achieve strong language modeling performance while restoring recall capability [41, 43, 64, 45]. We show that looped transformers open a *second* axis for such mixing: beyond varying the mixer along *depth*, we can also vary it across *loop iterations*. We explore both options (Figure 3) in following section 3.3.

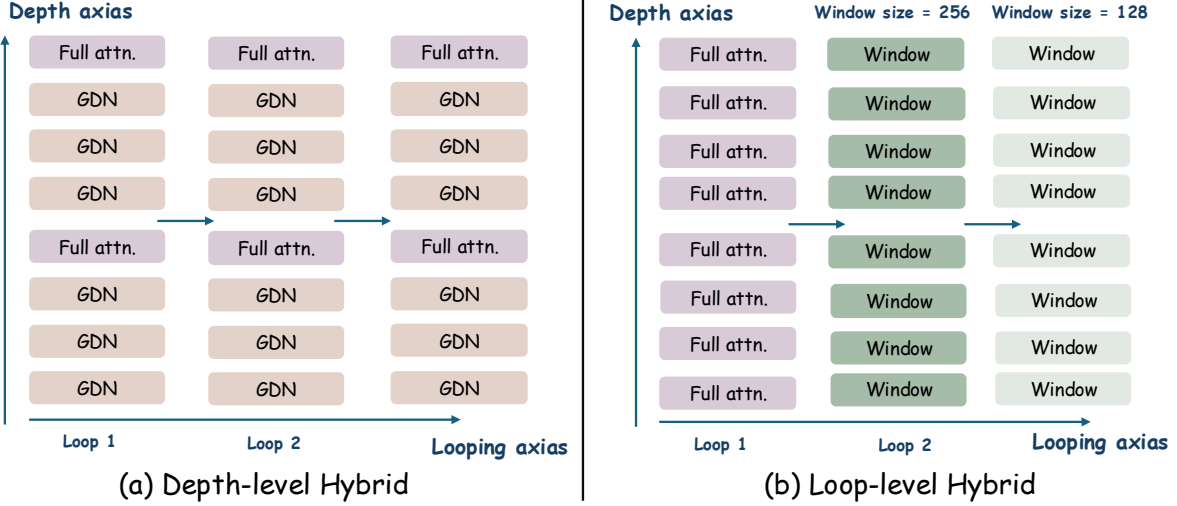


Figure 3: Two ways to hybridize LT2. **(a) Depth-level** interleaves full-attention layers among linear layers inside the shared block. **(b) Loop-level** varies the mixer across loop iterations, e.g. a full-attention loop first, then sliding-window loops with shrinking windows ($256 \rightarrow 128$).

3. Experiments

We organize the main experiments around four questions. First, we test whether LT2 is competitive at standard language-modeling scale (§ 3.1) and under realistic long-context retrieval (§ 3.7). We then ablate the hybrid-design choices: where hybridization is applied, how mixers are arranged along depth, and what mixer ratio is used (§ 3.3). We then study the SDPA output gate, which mitigates attention-sink accumulation under looped processing (§ 3.4). Additional, we also did experiments cover synthetic recall/state tracking, long-context efficiency, and training stability (§ 3.5, 3.2, and 3.6).

3.1. Language modeling

We pre-train all models on FineWeb-Edu [46] at 0.6B and 1.3B parameters with a 100B-token budget, using $T=4$ loops for every looped variant. The hybrid ratio is 1:4 for both (Full:Linear) and (GDN:DSA) variants. The full setup is in Appendix C. Table 2 summarizes the results. We provide a detailed efficiency comparison in Section 3.2.

Subquadratic Mixers Nearly Match the Full-Attention Loop. Looped GDN, KDA, and DSA all perform within roughly one average point of the Looped Transformer reference at both scales, while avoiding quadratic complexity. At the smaller 0.6B scale, looped GDN still slightly trails the Looped Transformer; however, at the larger 1.3B scale, it surpasses the Looped Transformer while preserving linear-time complexity. In the looped setting, we find that both gating and DPLR linear attention are important, with gating appearing to play an even larger role than the DPLR-style update formulation. In contrast, the looped pure DeltaNet variant is less stable in our study, which ultimately limits its performance.

A Linear-Sparse Hybrid Loop Matches the Full-Attention Loop at a Fraction of the Cost. Looped Hybrid (GDN+DSA), which contains no full attention, matches the full-attention reference at both scales (9.72 vs. 9.87 PPL at 1.3B). It also delivers the greatest efficiency speedup: a $2.9\times$ decode-throughput speedup at 32k context. We think this is an interesting hybrid setting where linear attention helps with global compression while sparse attention helps with exact KV position selection.

Table 2: Zero-shot downstream performance across two scales on FineWeb-Edu, $K=4$ loops. D-Gate = data-dependent gating; Δ = DPLR linear variants. **Cream** marks the best LT2 model without full attention. Best per column *within each scale* in **bold**, second-best underlined. Token budgets are relative to Chinchilla compute-optimal scaling [29].

Model	D-Gate	Δ	PPL($\downarrow$)	ARC-E	ARC-C	HellaS.	PIQA	WG	OBQA	SciQ	BoolQ	Avg.($\uparrow$)
0.6B parameters / 100B tokens (8$\times$ Chinchilla Ratio [29])												
Transformer	—	—	13.14	63.09	30.72	47.43	69.53	56.24	35.6	68.2	50.07	51.34
Looped Transformer (ref)	—	—	11.92	67.13	34.67	53.29	70.58	62.83	<u>38.2</u>	73.6	54.87	56.42
► LT2-linear attention												
Looped RetNet [59]	✗	✗	—				training diverged					
Looped HGRN2 [50]	✓	✗	14.59	59.82	27.93	43.17	67.34	52.13	33.4	65.2	48.53	49.69
Looped Mamba2 [14]	✓	✗	12.78	64.53	31.82	49.87	69.74	58.63	35.6	68.8	51.83	53.86
Looped DeltaNet [54, 72]	✗	✓	14.16	60.47	28.53	44.22	67.87	53.24	33.8	65.5	49.13	50.12
Looped GDN [70]	✓	✓	12.06	66.43	33.89	52.62	70.27	61.48	36.4	70.5	54.13	55.74
Looped KDA [63]	✓	✓	12.13	66.12	33.63	52.37	70.13	61.22	36.2	70.2	53.92	55.49
► LT2-sparse attention												
Looped Window	—	—	12.87	64.23	31.53	48.83	69.82	57.34	35.8	68.5	51.23	52.17
Looped NSA [75]	—	—	12.30	65.57	32.74	51.43	70.04	60.32	36.0	69.5	53.13	54.84
Looped DSA [15]	—	—	12.08	66.37	33.82	52.53	70.23	61.42	36.4	70.4	54.07	55.67
► Hybrid LT2 (linear/sparse/full permutation)												
Looped Hybrid (Full+Window)	—	—	12.24	65.32	32.13	51.23	69.86	58.42	36.0	69.2	53.13	54.43
Looped Hybrid (Full+DSA)	—	—	12.20	65.53	32.34	51.42	70.04	58.63	36.2	69.4	53.32	54.62
Looped Hybrid (Full+GDN)	✓	✓	11.43	69.82	37.34	55.83	72.62	64.61	38.9	73.3	57.74	58.65
Looped Hybrid (GDN+DSA)	✓	✓	<u>11.85</u>	<u>67.43</u>	<u>34.53</u>	<u>53.42</u>	<u>70.63</u>	<u>62.92</u>	37.0	71.2	<u>55.13</u>	<u>56.53</u>
1.3B parameters / 100B tokens (4$\times$ Chinchilla Ratio [29])												
Transformer	—	—	10.65	67.52	33.84	52.47	71.03	61.48	36.6	71.3	54.02	56.04
Looped Transformer (ref)	—	—	9.87	70.83	37.54	57.06	72.43	65.83	38.6	74.1	57.83	59.27
► LT2-linear attention												
Looped Mamba2 [14]	✓	✗	10.30	69.47	36.63	55.94	72.68	64.37	38.2	73.0	57.03	58.43
Looped GDN [70]	✓	✓	9.75	71.28	38.33	57.73	73.37	66.26	39.1	74.3	58.78	59.92
Looped KDA [63]	✓	✓	9.68	71.57	38.62	57.99	73.53	66.42	39.3	74.6	58.98	60.14
► LT2-sparse attention												
Looped Window	—	—	10.42	68.43	35.47	54.87	71.32	63.23	36.9	71.7	55.87	57.23
Looped NSA [75]	—	—	10.17	69.02	35.97	55.08	71.52	64.03	37.2	72.2	56.53	57.72
Looped DSA [15]	—	—	9.97	69.93	36.93	56.38	71.94	64.87	37.7	72.9	57.42	58.54
► Hybrid LT2 (linear/sparse/full permutation)												
Looped Hybrid (Full+Window)	—	—	9.84	70.93	37.12	56.68	73.12	64.34	38.8	73.3	58.56	59.13
Looped Hybrid (Full+DSA)	—	—	9.80	71.13	37.28	56.84	73.24	64.52	38.9	73.4	58.73	59.28
Looped Hybrid (Full+GDN)	✓	✓	9.12	74.82	41.63	61.04	75.93	69.52	41.3	75.4	62.04	62.89
Looped Hybrid (GDN+DSA)	✓	✓	<u>9.50</u>	<u>72.44</u>	<u>39.33</u>	<u>58.84</u>	<u>73.98</u>	<u>67.13</u>	<u>39.7</u>	<u>74.9</u>	<u>59.77</u>	<u>60.73</u>

Looped Hybrid (Full+GDN) Pushes to a New Pareto Frontier. This is the strongest configuration overall, improving the general language modeling performance at both scales (61.39 vs. 59.27 at 1.3B), with the largest gains on harder reasoning task. Since only a small fraction of layers are quadratic, it still yields a $\times 2.7$ decode speedup. Together, the two hybrids bracket the new Pareto frontier: one matches full attention at near-linear cost, the other exceeds it while staying markedly faster than the all-full-attention loop.

3.2. Efficiency at long context

We measure prefill and decode throughput for the four looped LT2 candidates from 1k to 32k tokens, at batch sizes $\{1, 2, 4, 8\}$, on a single H100 (80 GB) with FlashAttention-2 [13] for softmax attention and a fused chunkwise kernel for GDN. All variants use $T=4$ at matched parameter count. Open squares mark the last length each configuration fit in memory before going out-of-memory.

Two trends are visible across all four batch rows.

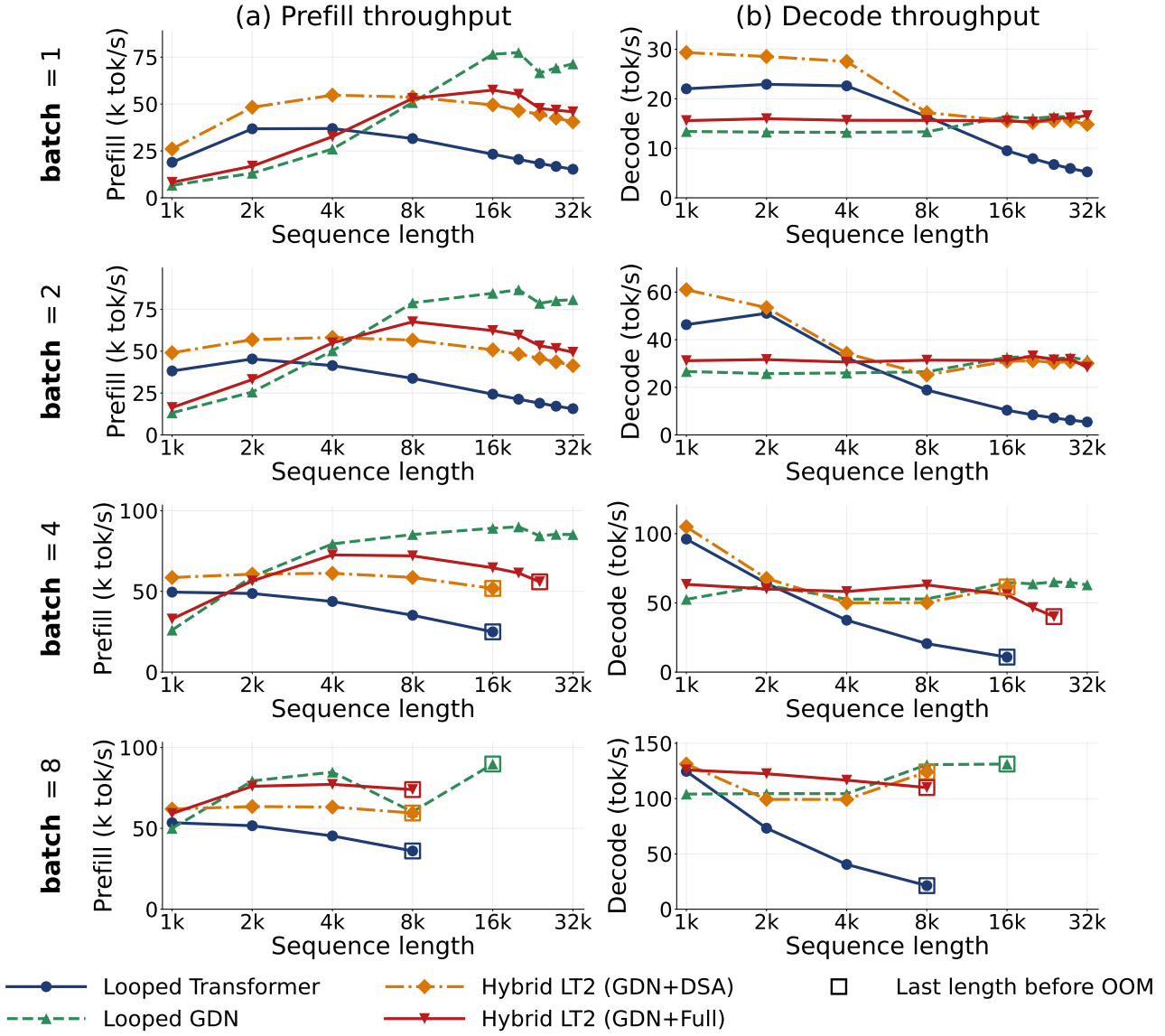


Figure 4: Efficiency at long context across batch sizes. Rows are batch size (1,2,4,8); columns are (a) prefill and (b) decode throughput vs. sequence length. Open squares mark the last sequence length each configuration reached before exhausting 80 GB of HBM. Looped GDN and Hybrid LT2 (GDN+Full) are the only variants that reach 32k at every batch size and that hold their decode throughput flat across the full range; Hybrid LT2 (GDN+DSA) tracks them closely thanks to its top- k KV reads.

Linear-time mixers eliminate the long-context decode cliff. Looped Transformer loses more than half of its decode throughput between 4k and 32k because its KV cache keeps growing per loop iteration. Looped GDN, Hybrid LT2 (GDN+Full), and Hybrid LT2 (GDN+DSA) all hold a flat decode rate across the entire range and across all batch sizes: at $bs=1$, 32k they decode roughly $3\times$ faster than the LT, and at $bs=8$ they reach 32k while the LT OOMs by 8k. The advantage compounds with batch size, since each extra batch element adds a fixed-size GDN state but a length-proportional KV cache.

Table 3: Hybrid LT2 ablations. 1.3B / $T=4$ / 100B FineWeb-Edu tokens. **Cream** marks the best row in each group. Avg. is the eight-task mean from Table 2.

Configuration	Full:GDN	Pattern / Schedule	PPL ($\downarrow$)	Avg. ($\uparrow$)
► (1) Hybrid ratio (depth-interleaved)				
Looped Transformer (ref)	1:0	—	9.87	59.27
Hybrid 1:1	1:1	interleave	9.41	60.92
Hybrid 1:4 (default)	1:4	interleave	9.31	61.39
Hybrid 1:6	1:6	interleave	9.36	61.07
Hybrid 1:12	1:12	interleave	9.74	59.51
Looped GDN	0:1	—	10.02	58.42
► (2) Hybrid pattern (ratio fixed at 1:4, depth-level)				
Bookend	1:4	Full at top & bottom, GDN in middle	9.27	61.52
Interleave (default)	1:4	every 5th layer is Full	9.31	61.39
Front-loaded	1:4	all Full layers at the bottom of the stack	9.45	60.61
Back-loaded	1:4	all Full layers at the top of the stack	9.53	60.43
► (3) Hybridization level (matched parameters)				
Random sample + majority vote ($K=5$)	1:4	resample 1/5 Full per step; vote at eval	9.26	61.55
Depth-level (default)	1:4	per-layer Full / GDN interleave	9.31	61.39
Loop-level coarse→fine	—	Full → SWA-512 → SWA-256 → SWA-128	9.36	60.71
Loop-level fine→coarse	—	SWA-128 → SWA-256 → SWA-512 → Full	9.42	61.10

Linear-time mixers also extend the OOM frontier. As batch grows, the Looped Transformer OOMs progressively earlier (bs=4: cap near 16k, bs=8: cap near 8k), while Looped GDN reaches 32k at every batch size and Hybrid LT2 (GDN+Full) reaches 32k up to bs=8. Hybrid LT2 (GDN+DSA) sits between the two, since DSA still maintains a KV cache for top- k selection but only reads a small slice of it per query. In practice this is the difference between serving long context at a useful batch size and not.

3.3. Ablation: hybrid ratio, pattern, and hybridization level

The hybrid LT2 in Section 3.1 fixes three design choices at once: *how much* attention sits in the loop, *where* it sits along depth, and *at what level* mixers are mixed. In this section we did a careful ablation study over this (Table 3).

Ratio: clean inverse-U with the optimum at 1:4. Sweeping the Full:GDN ratio between Looped Transformer (1:0) and Looped GDN (0:1), the interior traces a clean inverse-U with 1:4 on top. Too much attention crowds out the recurrent regularization documented in Section 3.5; too little starves the loop of precise retrieval. 1:4 is the smallest amount of attention that still recovers full retrieval quality, matching standard hybrid Transformer baselines [38].

Pattern: spreading beats concentrating. At fixed 1:4, bookend (Full at top and bottom, GDN in the middle) slightly edges out the uniform interleave, hinting at a small benefit from attention at both the input encoding and the final read-out. Concentrating the attention layers at one end — front-loaded or back-loaded — loses more than 0.7 points of average accuracy. Maybe the takeaway is that any reasonable spread along depth is much better than any concentration.

Level: across-iteration heterogeneity does not help. We try three loop-level schedules in place of depth-level mixing: *coarse-to-fine* (Full→SWA-512→SWA-256→SWA-128), *fine-to-coarse* (the reverse), and a stochastic baseline that resamples a 1:4 depth-level hybrid every step and majority-votes $K=5$ samples at eval. Coarse-to-fine wins on PPL but loses on downstream, it narrows the receptive field on the

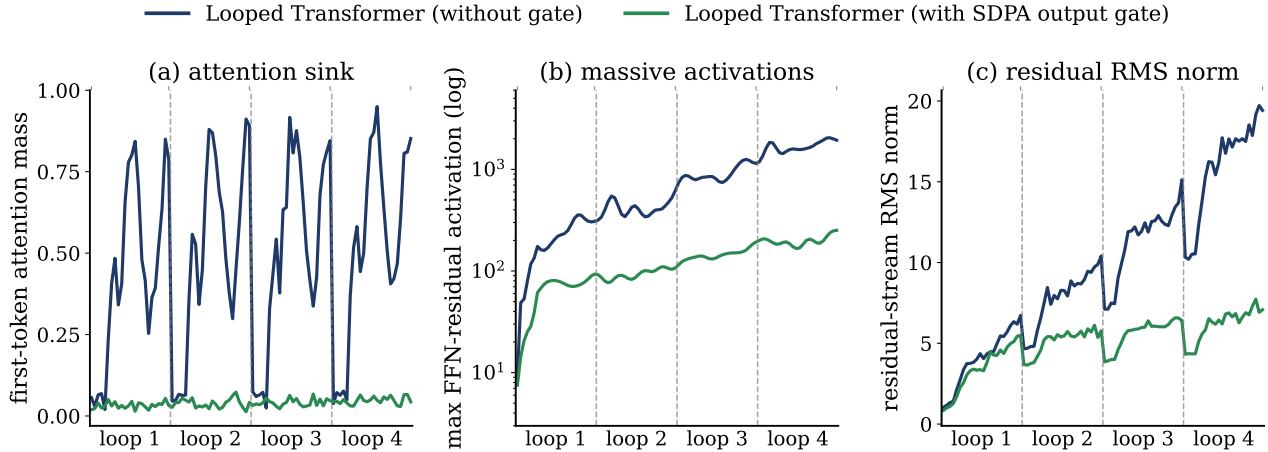


Figure 5: Unrolled diagnostics for the Looped Transformer ($T=4$, 24 layers). The x-axis runs through the unrolled computation, with dashed lines marking loop boundaries. **(a)** First-token attention mass forms a sawtooth that intensifies each loop — the sink is re-injected rather than reset. **(b)** Max FFN-residual activation follows the same compounding pattern (log scale). **(c)** Residual-stream RMS norm grows with both within-loop depth and across loop iterations. The SDPA output gate flattens (a)/(b) and substantially mitigates — though does not eliminate — the across-loop growth in (c).

final iteration over-fits local statistics. Fine-to-coarse inverts the trade. Random-vote is the best overall but at $5\times$ inference compute, which is hard to justify against the simple fixed interleave.

3.4. Ablation: attention sinks and the SDPA output gate

A natural concern with weight-shared loops is that pathologies of the underlying attention block (in particular, the *attention sink* [68]) where a small set of tokens absorb a disproportionate share of softmax mass [58] may compound across loop iterations: the same softmax block is re-applied to a residual stream that already carries the sink from the previous pass. Gated Attention [51] show that a head-specific sigmoid gate after Scaled Dot-Product Attention (SDPA) eliminates the sink in standard transformers. We ask the same question for our looped models and adopt the same fix, applied inside the looped block so W_θ is reused on every iteration. We add this gate to the three LT2 variants, keep the FFN width matched in parameter count, and re-train at $1.3B/T=4$ on 100B FineWeb-Edu tokens.

The sink is real and compounds across loops. Figure 5 unrolls the Looped Transformer along the trajectory (loop 1, layer 1) $\rightarrow$ (loop 1, layer 24) $\rightarrow$ (loop 2, layer 1) $\rightarrow \dots$. The first-token attention mass traces a sawtooth that *intensifies across loops*: the sink learned in loop t is re-injected into loop $t+1$ rather than reset, so each successive iteration starts already biased toward the sink. The maximum residual activation follows the same compounding pattern, consistent with the established literature [58]. And the residual-stream RMS norm grows along both within-loop depth and across loop iterations, reaching ~ 20 times by the end of loop 4.

Table 4: Effect of the SDPA output gate on the three softmax-containing LT2 variants. $1.3B / T=4$ / 100B tokens. We report the mean over the eight zero-shot benchmarks of Table 2.

Model	Gate	PPL ($\downarrow$)	Avg. ($\uparrow$)
Looped Transformer	—	9.87	59.27
	✓	9.39	60.70
	Δ	-0.48	+1.43
LT2-Hybrid (Full+GDN)	—	9.31	61.39
	✓	9.03	62.33
	Δ	-0.28	+0.94
LT2-Hybrid (GDN+DSA)	—	9.72	59.23
	✓	9.53	59.96
	Δ	-0.19	+0.73

Overall, attention sinks and massive activations

are not artifacts of standard transformers alone. weight-shared loops mildly amplify them by re-applying the same softmax to a residual that already carries the sink. A single head-specific sigmoid gate inside the loop suppresses the compounding and yields a small but consistent improvement on every softmax-containing LT2 variant. We recommend adopting this gate in all such variants in looped setting.

3.5. Training stability

A practical concern with looped models is that repeatedly applying the same block can amplify activations and destabilize optimization. We track the language-modeling loss and global gradient norm throughout pre-training and find that the choice of mixer inside the loop has a pronounced effect on stability.

Gating and the delta rule keep the linear loop bounded.

Figure 6 compares the looped Transformer against four subquadratic mixers. Looped RetNet exhibits persistently large gradient norms and frequent spikes throughout training, consistent with its divergence in Table 2. Looped DeltaNet and Looped Mamba2 are noticeably better but still display occasional spikes that propagate into the loss. In contrast, Looped GDN tracks below the Looped Transformer in gradient norm for essentially the entire run and produces the smoothest loss curve of any variant. Two ingredients appear to matter: a data-dependent gate, which lets the recurrence forget stale state instead of letting it accumulate across iterations, and the delta rule, which bounds updates to the recurrent memory. Mixers that have only one of the two (Mamba2 has gating without the delta rule; DeltaNet has the delta rule with weaker gating) are stable but visibly noisier than GDN, while RetNet, which has neither, is unstable.

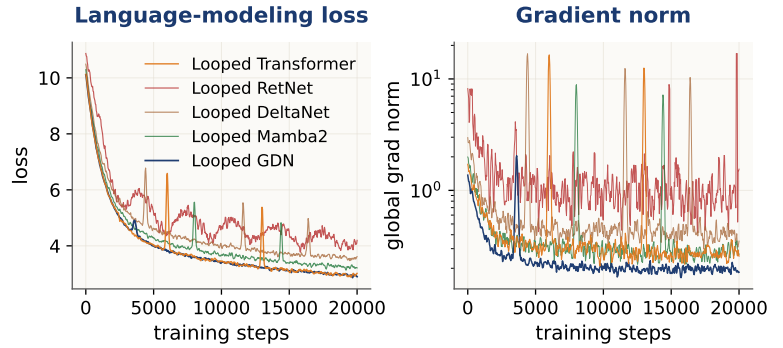


Figure 6: Looped GDN trains with the smoothest loss and the smallest gradient norms across all linear and full-attention variants; Looped RetNet, which lacks both data-dependent gating and a delta rule, diverges.

Sparse attention is stable but slightly less capable.

Figure 7 reports the same diagnostics for sparse-attention loops. All three sparse variants (Window, NSA, DSA) train smoothly: their gradient norms sit at or below the Looped Transformer for the entire run, and none of them shows the sharp spikes that occasionally appear in the full-attention loop around the middle of training. The price is a small but consistent gap in language-modeling loss — restricting each iteration to a sparse receptive field caps the per-loop computation and slows convergence relative

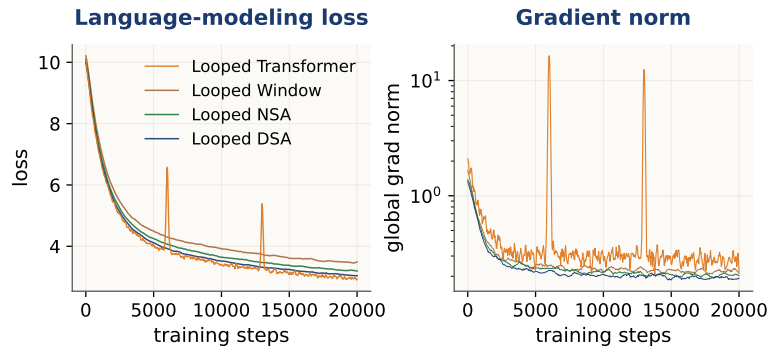


Figure 7: Sparse looped variants train without the spikes seen in the full-attention loop, but reach a slightly higher final loss than the Looped Transformer.

to dense attention. Among the sparse choices, Looped DSA is the strongest, which is why we adopt it as the sparse component of LT2 in the remainder of the paper.

Hybrid mixers combine stability and capability.

Figure 8 shows that the two hybrid configurations inherit the best of both worlds. Looped Hybrid (GDN+DSA) and Looped Hybrid (Full+GDN) both track the Looped Transformer in loss from the very beginning of training and pull slightly ahead by the end, while their gradient norms remain consistently smaller and free of the spikes that the full-attention loop occasionally produces. Pairing a recurrent mixer with either sparse or dense attention thus appears to regularize the loop: the linear branch keeps the gradient norm bounded across iterations, while the attention branch supplies the precise retrieval that pure linear models lack.

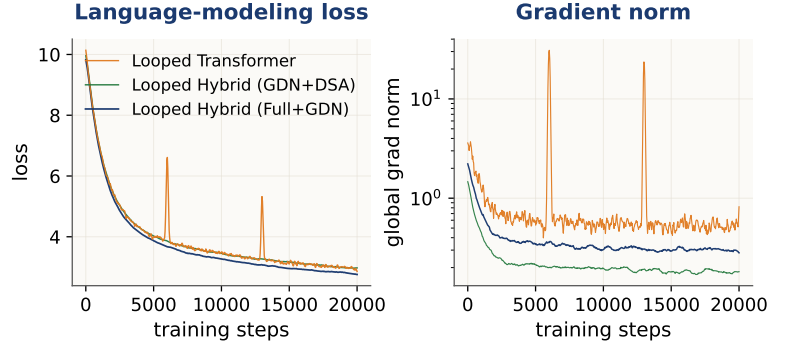


Figure 8: Both hybrid variants match or beat the Looped Transformer in loss while producing smaller and smoother gradient norms throughout training.

Across all three comparisons the same picture emerges. Mixers with data-dependent gating and a delta rule (GDN, and the hybrids that include it) train more stably than vanilla full attention under looping, and sparse attention, while less expressive, never destabilizes. This motivates the two LT2 instantiations used in the rest of the paper: *LT2-sparse* (Looped Hybrid with DSA) when stability is the priority, and *LT2-linear* (Looped Hybrid with GDN) when capability is.

3.6. Synthetic Tasks: State-tracking + Recall

State tracking and long-range retrieval are usually treated as opposite stress tests: state tracking favors recurrent depth, while retrieval favors precise attention to a long history. The first synthetic experiment we use to probe LT2 puts both pressures on the same task. We follow the *state-based recall* construction of Olmo-hybrid [43].

```
bits = [1,0,1,1,0,0,1,0,1,1,0,1,0,1,1,0,0,1,0,1,0,1,1,0,1,0,1,1,0,0,1,0]      recall target
a, b, c, d, e = 11, 27, 4, 19, 30      pointer state: (a,b,c,d,e)=(11,27,4,19,30)
a, c = c, a;      swap a↔c  a=4, c=11
b, d = d, b;      swap b↔d  b=19, d=27
a, e = e, a;      swap a↔e  a=30, e=4
c, d = d, c;      swap c↔d  c=27, d=11
d, e = e, d;      swap d↔e  d=4, e=11
a, b = b, a;      swap a↔b  a=19, b=30
b, c = c, b;      swap b↔c  b=27, c=30
a, d = d, a;      swap a↔d  a=4, d=19
assert bits[a] == ?      final query: a=4, so target = bits[4] = 0
```

Task and setup. Each example is a short Python-like program (see below for $m=32$, $n=8$): a bit array `bits` of length m is written, five variables `a–e` are bound to five distinct indices in $[0, m)$, then n swap lines are emitted with an `assert bits[a] == ?` after every swap. The model is supervised on each `?` token.

The task combines *long-range recall* (fetch bits $[\cdot]$ from $\Theta(m)$ tokens earlier, hard for compressive RNNs) with *state tracking* (apply the running sequence of transpositions to the pointer, hard for fixed-depth Transformers in TC^0). We tie $n=m$ and grow them together along the curriculum $\{8, 16, 32, 64, 128, 256\}$; a model advances once eval accuracy reaches 0.90 within a 100k-step budget. The headline metric is $n_{\max}$, the largest $n=m$ solved. All models share a 4-layer, 256-wide, 4-head backbone (RoPE); loop variants share weights across T iterations of this backbone, $T=1$ is the standard non-loop model. We train with AdamW (peak LR 3×10^{-4} , batch 32).

Effect of looping. Figure 9 reports the highest stage solved for every (architecture, T). The striking pattern is that *looping helps subquadratic mixers more than it helps full attention*. Looped Transformer and Looped Full+Window plateau at stage 4 ($n_{\max}=64$) and never reach stage 5 at any T . By contrast, three subquadratic variants — Looped NSA, Looped GDN+Window, and Looped GDN+NSA — all reach stage 5 ($n_{\max}=128$); Looped Full+GDN does too, but only because half its block is already linear-time GDN. Looped GDN+Window is the most dramatic: stage 3 at $T \leq 4$, stage 5 at $T=8$.

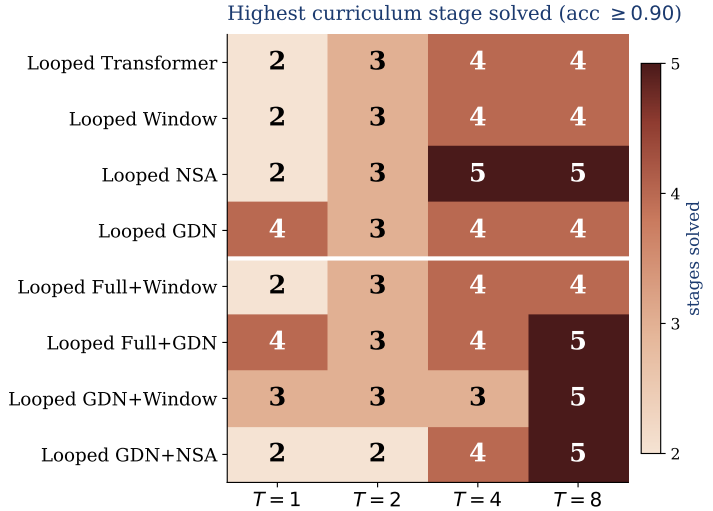


Figure 9: Highest curriculum stage s solved wrt. loop count T . Pure mixers above the white line, hybrids below.

Comparison across architectures. The reference point is the Looped Transformer. Relative to it, several subquadratic mixers gain *both* expressive power and recall on this joint task. Looped Transformer caps at stage 4 at all T , but Looped NSA, Looped GDN+Window, and Looped GDN+NSA all reach stage 5 — a doubling of $n_{\max}$ over the global-attention baseline at the same parameter budget. Among these, Looped GDN+NSA and GDN+Window are the fully linear-time models to solve it, consistent with the great language modeling performance discussed above.

3.7. Realistic recall and long-context retrieval

We now turn to realistic long-context recall, where the model must retrieve specific facts from natural text far longer than fits comfortably into a recurrent state. We follow the evaluation protocol of Mamba-3 [38]. All models are at the 1.3B scale. pure models stack a single mixer family throughout, while hybrids interleave that mixer with full attention in a fixed 4:1 ratio, and looped variants share weights across $T=4$ iterations of the corresponding non-looped backbone at matched parameter count. We evaluate two complementary suites: knowledge-style recall on SWDE [1], SQuAD [52], FDA [1], TriviaQA [33], Natural Questions [37], and DROP [17] at 2048 tokens, and precise needle-in-a-haystack retrieval (NIAH-Single-1/2/3) [30] at 1024, 2048, and 4096 tokens, where the 4096 column is the most informative since all models were pre-trained at 2048 and must extrapolate.

Three findings stand out (Table 5). Looping consistently improves the underlying mixer at matched parameter count: Looped Transformer, Looped GDN, and Looped Mamba-2 each gain roughly 2–4 points on average over their non-looped counterparts on the knowledge-style suite, with task-level fluctuations in either direction (some columns such as TQA are already saturated, while FDA and NQ benefit more

from extra iterations), and they retain the qualitative NIAH behavior of their base versions – recurrent backbones extrapolate gracefully to 4096 while dense-attention ones do not.

Looped Hybrid (GDN+DSA) tracks the Looped Transformer closely on the knowledge suite despite containing no quadratic component, and additionally extrapolates substantially better at NIAH-4096.

Looped Hybrid (Full+GDN) is the strongest configuration overall, improving over the Looped Transformer

Table 5: Long-context evaluation at 1.3B parameters. Knowledge-style benchmarks (SWDE–DROP) are evaluated at 2048 tokens; NIAH-Single-1/2/3 at 1024, 2048, and 4096 tokens (models pre-trained at 2048 must extrapolate to 4096). Pure models use a single mixer; hybrid models interleave with full attention in a 1:1 ratio. Looped variants share weights across $T=4$ iterations of the corresponding non-looped backbone. **Bold** marks the best result per column; underline marks the second best.

Model (1.5B) Context Length	SWDE	SQD.	FDA	TQA	NQ	DROP	NIAH-Single-1			NIAH-Single-2			NIAH-Single-3		
	2048						1024	2048	4096	1024	2048	4096	1024	2048	4096
Transformer	48.9	46.6	58.4	67.5	31.7	26.4	100.0	100.0	0.0	92.2	100.0	0.0	98.6	99.4	0.0
GDN	32.7	40.0	28.3	63.5	25.7	24.5	100.0	100.0	99.8	100.0	93.8	49.8	83.8	68.4	34.2
Mamba-2	30.7	39.1	23.7	64.3	25.1	28.5	100.0	99.6	62.0	100.0	53.8	11.8	95.8	87.4	13.4
Looped Transformer	52.8	49.4	61.7	68.2	33.6	28.1	100.0	100.0	0.0	94.6	100.0	0.0	99.2	99.8	0.0
Looped GDN	34.9	41.8	30.6	64.7	27.0	25.9	100.0	100.0	99.8	100.0	<u>96.4</u>	53.2	85.6	71.0	35.8
Looped Mamba-2	33.9	40.5	25.8	65.1	26.8	<u>29.7</u>	100.0	100.0	65.7	100.0	57.1	13.5	96.2	88.1	16.2
Looped Hybrid (GDN+DSA)	51.6	48.0	60.4	66.9	33.0	28.4	100.0	100.0	91.4	100.0	100.0	<u>77.6</u>	100.0	<u>99.6</u>	<u>60.3</u>
Looped Hybrid (Full+GDN)	53.1	<u>48.9</u>	62.0	<u>67.8</u>	34.0	30.2	100.0	100.0	<u>93.5</u>	100.0	100.0	81.0	<u>99.8</u>	99.8	63.7

on the average of the knowledge-style benchmarks while degrading more gracefully than the Looped Transformer at NIAH-4096 thanks to its GDN branch. As before, individual cells fluctuate across this broad suite, but the ordering on the aggregate is stable and matches the picture from Section 3.1: the linear–sparse loop (LT2-sparse) recovers full-attention quality, and the full–linear loop (LT2-linear) extends the Pareto frontier.

4. Distilling Looped Transformers into LT2

A natural follow-up to the from-scratch results above is whether LT2 can also be reached *post-hoc*, by replacing most of a pre-trained Looped Transformer’s quadratic attention with linear-time attention variants. We extend the hybrid-distillation recipe of non-looped models [40] to the looped setting with the full attention ratio of 25%. The overall results is shown in Figure 10.

4.1. Algorithm

We follow the two-stage RADLADS-style pipeline [40]. The teacher f_{θ_T} is Ouro-1.4B [76] which is a standard looped transformer. The student f_{θ_S} shares the teacher’s embedding, FFN, and norm parameters but uses GDN as token mixer outside a small kept-attention set $\mathcal{F} \subseteq \{1, \dots, N\}$. At loop iteration $\tau \in \{1, \dots, T\}$ each network emits logits $z_{T/S}^{(\tau)} \in \mathbb{R}^{L \times V}$, where L is the sequence length and V is the vocabulary size.

Stage 1 (linear pre-alignment). With $\mathcal{F} = \emptyset$, we align each GDN block to the corresponding teacher attention output via an MSE loss on the residual stream (100M tokens, length 512).

Stage 2 (hybrid logit distillation). We restore the $|\mathcal{F}| = 6$ softmax layers selected by the KL-guided selector to pick top full attention layers and distill the teacher’s logits. The looped setting introduces a

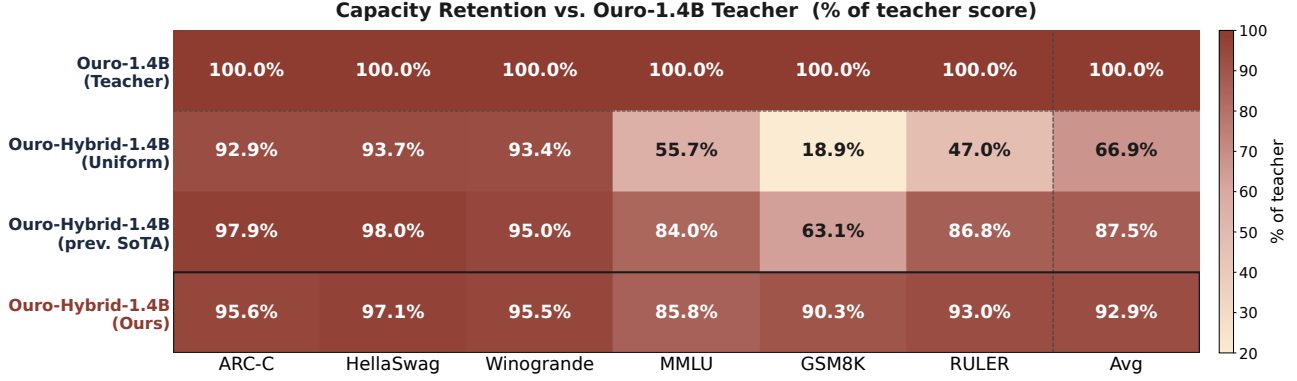


Figure 10: Capability retention of distilled Ouro-Hybrid-1.4B variants. *Ouro-Hybrid (Uniform)* interleaves linear and full-attention layers in a fixed interleaved pattern, while *Ouro-Hybrid (prev. SoTA)* selects the layers to retain full attention using the best method previously [40].

new design knob – the *per-loop* KL schedule – and we minimise

$$\mathcal{L}_{\text{KD}} = \sum_{\tau=1}^T w_t^{(\tau)} \text{KL} \left(\sigma_{\text{top-}k}(z_{\mathcal{T}}^{(\tau)} / T_{\text{kd}}) \parallel \sigma_{\text{top-}k}(z_{\mathcal{S}}^{(\tau)} / T_{\text{kd}}) \right), \quad (7)$$

where $\sigma_{\text{top-}k}$ renormalises the softmax over the teacher’s top- k tokens and $w_t \in \Delta^{T-1}$ controls how much supervision each loop receives at step t . We progressively warm up the loop-level supervision, set uniform weights to supervise equally per loop for half of the training steps, and then switch to final-output supervision only (600M tokens, length 4096).

Stage 3 (Long-context continuation). We then extend Stage 2 with a continuation phase on 35K OpenThoughts-v3 [28] with 32k sequence length, using the same KD loss in Eq. (7) and a constant LR (600M tokens, length 32768).

4.2. Results

Distilling pre-trained full-attention models into linear-time variants remains non-trivial. We find that the primary driver of performance on general and mathematical benchmarks is the composition of the distillation data. Stages 1 and 2 utilize general datasets like DCLM [39], resulting in high scores on commonsense downstream tasks. Integrating reasoning-specific data like OpenThoughts in Stage 3

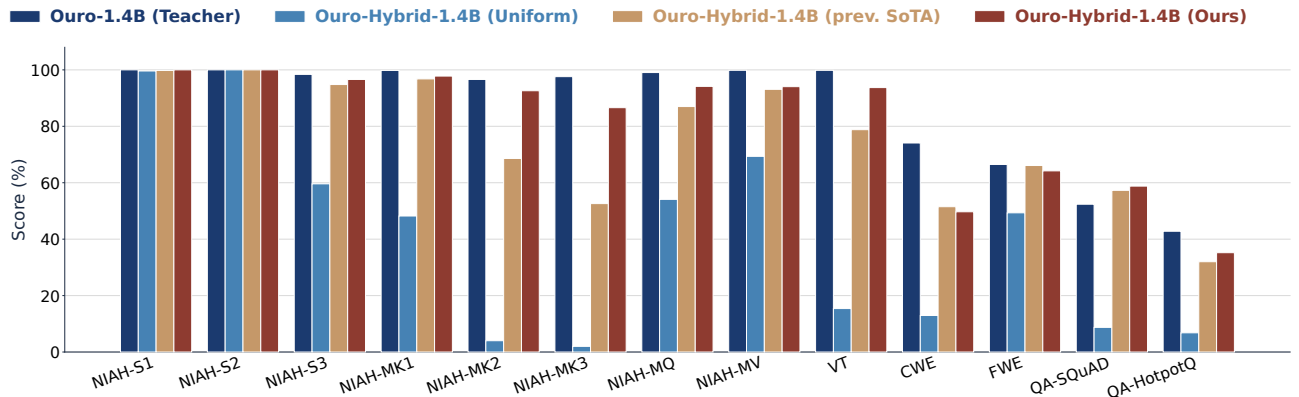


Figure 11: Ruler subtask performance for different distillation models. The key task differences lie in multi-key retrieval, which benefits more from per-loop supervision.

significantly narrows the gap in mathematical reasoning. Two factors are important from an algorithmic perspective. As shown in Figure 11, progressive length expansion is essential for maintaining long-context performance. Furthermore, per-loop supervision provides a more stable gradient signal than supervising the final loop alone. We recommend this multi-stage recipe for researchers distilling looped architectures.

Finally, we compare our distilled model against industry-level small language models (Figure 1), demonstrating highly competitive performance. We will release the full model checkpoints to the public to foster further research into efficient, high-capability small models.

5. Related Work

Looped Transformers and the path to scalable recursion. Universal Transformers [16] reintroduced depth-wise recurrence to the Transformer by tying the weights of every layer [5] and iterating the same block for a fixed or adaptive number of steps. Early follow-ups showed that this simple inductive bias improves systematic generalization on compositional benchmarks [11] and that, more broadly, parameter sharing across layers is a viable design choice rather than a curiosity [60]. On the theoretical side, looped architectures are Turing-complete under mild assumptions [49] and can be programmed to emulate iterative algorithms such as multi-step gradient descent [23, 69, 21, 20, 18], which has been formalized as a “latent thought” view of looping where each pass refines an internal computation [53, 25, 6, 8]. This perspective has driven a recent wave of recursion-centric reasoning models, including HRM [66], TRM [32], the Universal Reasoning Model [19], and looped language models trained at scale [76]. The central obstacle, however, is scalability: looping the same block T times multiplies compute by T without adding parameters, so naive Universal Transformers underperform standard Transformers under matched FLOPs [62, 48]. Three lines of work directly target this efficiency gap. The first injects sparsity and mixture-of-experts into the shared block so that capacity grows without proportional compute, as in the Sparse Universal Transformer [61], MoEUT [12], Mixture of Universal Experts [10], and parameter-efficient FFN reuse [44]. The second relaxes strict weight tying with low-rank deltas so that each iteration can specialize cheaply [3]. The third, most directly in the spirit of Adaptive Computation Time, allocates a variable number of recursion steps per token: Mixture-of-RecurSIONs learns dynamic per-token depths in a token-level routing framework [4], while elastic and depth-recurrent variants extend this idea to vision and attention-aware latent reasoning [26, 36, 74, 56]. Recent work has further accelerated inference of recurrent-depth models through parallel sampling [22]. Our work continues this trajectory, focusing on how to make looped computation scale predictably under a fixed compute budget.

Subquadratic attention. A parallel line of research replaces softmax attention with sequence mixers whose cost is linear, or near-linear, in the sequence length. Linear attention [35] expresses attention as a kernel feature map and reformulates inference as a recurrent state update, which is interpreted as fast-weight programming and trace back to earlier work on associative fast weights [55, 31, 2]. From this foundation a family of efficient recurrences has emerged, including RetNet [59], Gated Linear Attention [71], HGRN2 [50], DeltaNet and its parallel and gated variants [72, 70], the SSM-attention duality of Mamba-2 [14] and its successor Mamba-3 [38], and RWKV-7 [47]. Recent work has also addressed the limited state-tracking capacity of these recurrences through negative eigenvalues [27] and Householder products [57]. Because pure linear-attention models still lag softmax attention on tasks requiring exact recall [1], a complementary line interleaves linear and softmax layers in hybrid stacks such as Jamba [41], Kimi Linear [63], and Olmo Hybrid [43], or distills pretrained Transformers into hybrid or linear successors [24, 40]. A third strand keeps softmax attention but enforces sparsity to reduce its quadratic cost, ranging from attention sinks for streaming inference [68] to natively trainable block-sparse patterns [75, 9, 15]. These approaches are largely orthogonal to depth-wise recursion: they

reduce the cost of a single forward pass, whereas looping reuses parameters across passes. Combining the two is a natural direction, and one we explore in this work.

6. Conclusion

We presented LT2, a family of linear-time looped Transformers that replace the quadratic token-mixing bottleneck in looped architectures with linear, sparse, and hybrid attention mechanisms. In particular, our hybrid variants recover or exceed the quality of full-attention looped Transformers while substantially improving inference efficiency. These results suggest that efficient token mixers can make recursive depth a practical scaling axis for future language models.

Limitations. Two directions remain unexplored. First, we study depth-level hybridization and simple loop-level schedules but do not investigate full loop-level hybridization, where different iterations could use distinct attention families rather than only varying masks. Second, we do not design explicit cross-loop recurrent state carry mechanisms; principled state-sharing across loops may further improve long-context modeling, memory reuse, and compute efficiency.

References

- [1] S. Arora, S. Eyuboglu, M. Zhang, A. Timalsina, S. Alberti, D. Zinsley, J. Zou, A. Rudra, and C. Ré. Simple linear attention language models balance the recall-throughput tradeoff, 2025.
- [2] J. Ba, G. Hinton, V. Mnih, J. Z. Leibo, and C. Ionescu. Using fast weights to attend to the recent past, 2016.
- [3] S. Bae, A. Fisch, H. Harutyunyan, Z. Ji, S. Kim, and T. Schuster. Relaxed recursive transformers: Effective parameter sharing with layer-wise lora, 2025.
- [4] S. Bae, Y. Kim, R. Bayat, S. Kim, J. Ha, T. Schuster, A. Fisch, H. Harutyunyan, Z. Ji, A. Courville, et al. Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation. In *The Thirty-Ninth Annual Conference on Neural Information Processing Systems*. NeurIPS, 2025.
- [5] S. Bai, J. Z. Kolter, and V. Koltun. Deep equilibrium models, 2019.
- [6] H. Blayney, Álvaro Arroyo, J. Obando-Ceron, P. S. Castro, A. Courville, M. M. Bronstein, and X. Dong. A mechanistic analysis of looped reasoning language models, 2026.
- [7] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language models are few-shot learners, 2020.
- [8] G. Chen, D. Liu, and J. Shao. Loop as a bridge: Can looped transformers truly link representation space and natural language outputs?, 2026.
- [9] L. Chen, D. Xu, C. An, X. Wang, Y. Zhang, J. Chen, Z. Liang, F. Wei, J. Liang, Y. Xiao, and W. Wang. Powerattention: Exponentially scaling of receptive fields for effective sparse attention, 2025.
- [10] Y. Chen, N. Gu, J. Shang, Z. Zhang, Y. Feng, J. Sheng, T. Liu, S. Wang, Y. Sun, H. Wu, and H. Wang. Mixture of universal experts: Scaling virtual width via depth-width transformation, 2026.
- [11] R. Csordás, K. Irie, and J. Schmidhuber. The devil is in the detail: Simple tricks improve systematic generalization of transformers. In M.-F. Moens, X. Huang, L. Specia, and S. W.-t. Yih, editors,

- Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 619–634, Online and Punta Cana, Dominican Republic, Nov. 2021. Association for Computational Linguistics.
- [12] R. Csordás, K. Irie, J. Schmidhuber, C. Potts, and C. D. Manning. Moeut: Mixture-of-experts universal transformers. *Advances in Neural Information Processing Systems*, 37:28589–28614, 2024.
 - [13] T. Dao. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023.
 - [14] T. Dao and A. Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality, 2024.
 - [15] DeepSeek-AI, A. Liu, A. Mei, B. Lin, B. Xue, B. Wang, B. Xu, B. Wu, B. Zhang, C. Lin, C. Dong, C. Lu, C. Zhao, C. Deng, C. Xu, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, E. Li, F. Zhou, F. Lin, F. Dai, G. Hao, G. Chen, G. Li, H. Zhang, H. Xu, H. Li, H. Liang, H. Wei, H. Zhang, H. Luo, H. Ji, H. Ding, H. Tang, H. Cao, H. Gao, H. Qu, H. Zeng, and et al. Deepseek-v3.2: Pushing the frontier of open large language models, 2025.
 - [16] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and Łukasz Kaiser. Universal transformers, 2019.
 - [17] D. Dua, Y. Wang, P. Dasigi, G. Stanovsky, S. Singh, and M. Gardner. Drop: A reading comprehension benchmark requiring discrete reasoning over paragraphs, 2019.
 - [18] Y. Fan, Y. Du, K. Ramchandran, and K. Lee. Looped transformers for length generalization. In *The Thirteenth International Conference on Learning Representations*.
 - [19] Z. Gao, L. Chen, Y. Xiao, H. Xing, R. Tao, H. Luo, J. Zhou, and B. Dai. Universal reasoning model, 2025.
 - [20] K. Gatmiry, N. Saunshi, S. J. Reddi, S. Jegelka, and S. Kumar. On the role of depth and looping for in-context learning with task diversity.
 - [21] K. Gatmiry, N. Saunshi, S. J. Reddi, S. Jegelka, and S. Kumar. Can looped transformers learn to implement multi-step gradient descent for in-context learning? In *International Conference on Machine Learning*, pages 15130–15152. PMLR, 2024.
 - [22] J. Geiping, X. Yang, and G. Su. Efficient parallel samplers for recurrent-depth models and their connection to diffusion language models, 2025.
 - [23] A. Giannou, S. Rajput, J.-y. Sohn, K. Lee, J. D. Lee, and D. Papailiopoulos. Looped transformers as programmable computers. In *International Conference on Machine Learning*, pages 11398–11442. PMLR, 2023.
 - [24] D. Goldstein, E. Alcaide, J. Lu, and E. Cheah. Radlads: Rapid attention distillation to linear attention decoders at scale, 2026.
 - [25] Z. Gong, Y. Liu, and J. Teng. What makes looped transformers perform better than non-recursive ones, 2026.
 - [26] S. Goyal, S. Agrawal, G. G. Anil, P. Jain, S. Paul, and A. Kusupati. Elt: Elastic looped transformers for visual generation, 2026.
 - [27] R. Grazzi, J. Siems, A. Zela, J. K. H. Franke, F. Hutter, and M. Pontil. Unlocking state-tracking in linear rnns through negative eigenvalues, 2025.

- [28] E. Guha, R. Marten, S. Keh, N. Raoof, G. Smyrnis, H. Bansal, M. Nezhurina, J. Mercat, T. Vu, Z. Sprague, A. Suvarna, B. Feuer, L. Chen, Z. Khan, E. Frankel, S. Grover, C. Choi, N. Muennighoff, S. Su, W. Zhao, J. Yang, S. Pimpalgaonkar, K. Sharma, C. C.-J. Ji, Y. Deng, S. Pratt, V. Ramanujan, J. Saad-Falcon, J. Li, A. Dave, A. Albalak, K. Arora, B. Wulfe, C. Hegde, G. Durrett, S. Oh, M. Bansal, S. Gabriel, A. Grover, K.-W. Chang, V. Shankar, A. Gokaslan, M. A. Merrill, T. Hashimoto, Y. Choi, J. Jitsev, R. Heckel, M. Sathiamoorthy, A. G. Dimakis, and L. Schmidt. Openthoughts: Data recipes for reasoning models, 2025.
- [29] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J. W. Rae, O. Vinyals, and L. Sifre. Training compute-optimal large language models, 2022.
- [30] C.-P. Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg. Ruler: What’s the real context size of your long-context language models?, 2024.
- [31] K. Irie, I. Schlag, R. Csordás, and J. Schmidhuber. Going beyond linear transformers with recurrent fast weight programmers, 2021.
- [32] A. Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks, 2025.
- [33] M. Joshi, E. Choi, D. S. Weld, and L. Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension, 2017.
- [34] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models, 2020.
- [35] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pages 5156–5165. PMLR, 2020.
- [36] J. Knupp, J. H. Metzen, J. Bohn, G. Groh, and K. Kersting. Depth-recurrent attention mixtures: Giving latent reasoning the attention it deserves, 2026.
- [37] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee, K. Toutanova, L. Jones, M. Kelcey, M.-W. Chang, A. M. Dai, J. Uszkoreit, Q. Le, and S. Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019.
- [38] A. Lahoti, K. Y. Li, B. Chen, C. Wang, A. Bick, J. Z. Kolter, T. Dao, and A. Gu. Mamba-3: Improved sequence modeling using state space principles, 2026.
- [39] J. Li, A. Fang, G. Smyrnis, M. Ivgi, M. Jordan, S. Gadre, H. Bansal, E. Guha, S. Keh, K. Arora, S. Garg, R. Xin, N. Muennighoff, R. Heckel, J. Mercat, M. Chen, S. Gururangan, M. Wortsman, A. Albalak, Y. Bitton, M. Nezhurina, A. Abbas, C.-Y. Hsieh, D. Ghosh, J. Gardner, M. Kilian, H. Zhang, R. Shao, S. Pratt, S. Sanyal, G. Ilharco, G. Daras, K. Marathe, A. Gokaslan, J. Zhang, K. Chandu, T. Nguyen, I. Vasiljevic, S. Kakade, S. Song, S. Sanghavi, F. Faghri, S. Oh, L. Zettlemoyer, K. Lo, A. El-Nouby, H. Pouransari, A. Toshev, S. Wang, D. Groeneveld, L. Soldaini, P. W. Koh, J. Jitsev, T. Kollar, A. G. Dimakis, Y. Carmon, A. Dave, L. Schmidt, and V. Shankar. Datacomp-lm: In search of the next generation of training sets for language models, 2025.
- [40] Y. Li, S. Yang, S. Tan, M. Mishra, R. Panda, J. Zhou, and Y. Kim. Distilling to hybrid attention models via kl-guided layer selection, 2025.

- [41] O. Lieber, B. Lenz, H. Bata, G. Cohen, J. Osin, I. Dalmedigos, E. Safahi, S. Meirom, Y. Belinkov, S. Shalev-Shwartz, O. Abend, R. Alon, T. Asida, A. Bergman, R. Glozman, M. Gokhman, A. Manevich, N. Ratner, N. Rozen, E. Schwartz, M. Zusman, and Y. Shoham. Jamba: A hybrid transformer-mamba language model, 2024.
- [42] W. Luo, Y. Li, R. Urtasun, and R. Zemel. Understanding the effective receptive field in deep convolutional neural networks, 2017.
- [43] W. Merrill, Y. Li, T. Romero, A. Svete, C. Costello, P. Dasigi, D. Groeneveld, D. Heineman, B. Kuehl, N. Lambert, C. Li, K. Lo, S. Malik, D. Matusz, B. Minixhofer, J. Morrison, L. Soldaini, F. Timbers, P. Walsh, N. A. Smith, H. Hajishirzi, and A. Sabharwal. Olmo hybrid: From theory to practice and back, 2026.
- [44] Y. Nie, K. Han, H. Li, H. Zhou, T. Guo, E. Wu, X. Chen, and Y. Wang. Versatilefn: Achieving parameter efficiency in llms via adaptive wide-and-deep reuse, 2026.
- [45] NVIDIA, :, A. Blakeman, A. Grattafiori, A. Basant, A. Gupta, A. Khattar, A. Renduchintala, A. Vavre, A. Shukla, A. Bercovich, A. Ficek, A. Shaposhnikov, A. Kondratenko, A. Bukharin, A. Milesi, A. Taghibakhshi, A. Liu, A. Barton, and et al. Nvidia nemotron 3: Efficient and open intelligence, 2025.
- [46] G. Penedo, H. Kydlíček, L. B. allal, A. Lozhkov, M. Mitchell, C. Raffel, L. V. Werra, and T. Wolf. The fineweb datasets: Decanting the web for the finest text data at scale, 2024.
- [47] B. Peng, R. Zhang, D. Goldstein, E. Alcaide, X. Du, H. Hou, J. Lin, J. Liu, J. Lu, W. Merrill, G. Song, K. Tan, S. Utpala, N. Wilce, J. S. Wind, T. Wu, D. Wuttke, and C. Zhou-Zheng. Rwkv-7 "goose" with expressive dynamic state evolution, 2025.
- [48] H. Prairie, Z. Novack, T. Berg-Kirkpatrick, and D. Y. Fu. Parcae: Scaling laws for stable looped language models, 2026.
- [49] J. Pérez, J. Marinković, and P. Barceló. On the turing completeness of modern neural network architectures, 2019.
- [50] Z. Qin, S. Yang, W. Sun, X. Shen, D. Li, W. Sun, and Y. Zhong. Hgrn2: Gated linear rnns with state expansion. *arXiv preprint arXiv:2404.07904*, 2024.
- [51] Z. Qiu, Z. Wang, B. Zheng, Z. Huang, K. Wen, S. Yang, R. Men, L. Yu, F. Huang, S. Huang, D. Liu, J. Zhou, and J. Lin. Gated attention for large language models: Non-linearity, sparsity, and attention-sink-free, 2025.
- [52] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang. Squad: 100,000+ questions for machine comprehension of text, 2016.
- [53] N. Saunshi, N. Dikkala, Z. Li, S. Kumar, and S. J. Reddi. Reasoning with latent thoughts: On the power of looped transformers, 2025.
- [54] I. Schlag, K. Irie, and J. Schmidhuber. Linear transformers are secretly fast weight programmers. In *International conference on machine learning*, pages 9355–9366. PMLR, 2021.
- [55] I. Schlag, K. Irie, and J. Schmidhuber. Linear transformers are secretly fast weight programmers, 2021.
- [56] W.-J. Shu, X. Qiu, R.-J. Zhu, H. H. Chen, Y. Liu, and H. Yang. Loopvit: Scaling visual arc with looped transformers, 2026.

- [57] J. Siems, T. Carstensen, A. Zela, F. Hutter, M. Pontil, and R. Grazzi. Deltaproduct: Improving state-tracking in linear rnns via householder products, 2025.
- [58] M. Sun, X. Chen, J. Z. Kolter, and Z. Liu. Massive activations in large language models, 2024.
- [59] Y. Sun, L. Dong, S. Huang, S. Ma, Y. Xia, J. Xue, J. Wang, and F. Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.
- [60] S. Takase and S. Kiyono. Lessons on parameter sharing across layers in transformers. In N. Sادات Moosavi, I. Gurevych, Y. Hou, G. Kim, Y. J. Kim, T. Schuster, and A. Agrawal, editors, *Proceedings of the Fourth Workshop on Simple and Efficient Natural Language Processing (SustainLP)*, pages 78–90, Toronto, Canada (Hybrid), July 2023. Association for Computational Linguistics.
- [61] S. Tan, Y. Shen, Z. Chen, A. Courville, and C. Gan. Sparse universal transformer. In H. Bouamor, J. Pino, and K. Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 169–179, Singapore, Dec. 2023. Association for Computational Linguistics.
- [62] Y. Tay, M. Dehghani, S. Abnar, H. Chung, W. Fedus, J. Rao, S. Narang, V. Tran, D. Yogatama, and D. Metzler. Scaling laws vs model architectures: How does inductive bias influence scaling? In H. Bouamor, J. Pino, and K. Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 12342–12364, Singapore, Dec. 2023. Association for Computational Linguistics.
- [63] K. Team, Y. Zhang, Z. Lin, X. Yao, J. Hu, F. Meng, C. Liu, X. Men, S. Yang, Z. Li, W. Li, E. Lu, W. Liu, Y. Chen, W. Xu, L. Yu, Y. Wang, Y. Fan, L. Zhong, E. Yuan, D. Zhang, Y. Zhang, T. Y. Liu, H. Wang, S. Fang, W. He, S. Liu, Y. Li, J. Su, J. Qiu, B. Pang, J. Yan, Z. Jiang, W. Huang, B. Yin, J. You, C. Wei, Z. Wang, C. Hong, Y. Chen, G. Chen, Y. Wang, H. Zheng, F. Wang, Y. Liu, M. Dong, Z. Zhang, S. Pan, W. Wu, Y. Wu, L. Guan, J. Tao, G. Fu, X. Xu, Y. Wang, G. Lai, Y. Wu, X. Zhou, Z. Yang, and Y. Du. Kimi linear: An expressive, efficient attention architecture, 2025.
- [64] Q. Team. Qwen3.5-omni technical report, 2026.
- [65] M. Videau, B. Y. Idrissi, D. Haziza, L. Wehrstedt, J. Copet, O. Teytaud, and D. Lopez-Paz. Meta Lingua: A minimal PyTorch LLM training library, 2024.
- [66] G. Wang, J. Li, Y. Sun, X. Chen, C. Liu, Y. Wu, M. Lu, S. Song, and Y. A. Yadkori. Hierarchical reasoning model, 2025.
- [67] G. Xiao. Why stacking sliding windows can’t see very far. <https://guangxuanx.com/blog/stacking-swa.html>, 2025.
- [68] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks, 2024.
- [69] L. Yang, K. Lee, R. D. Nowak, and D. Papailiopoulos. Looped transformers are better at learning learning algorithms. In *The Twelfth International Conference on Learning Representations*.
- [70] S. Yang, J. Kautz, and A. Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. *arXiv preprint arXiv:2412.06464*, 2024.
- [71] S. Yang, B. Wang, Y. Shen, R. Panda, and Y. Kim. Gated linear attention transformers with hardware-efficient training. In *Forty-first International Conference on Machine Learning*.
- [72] S. Yang, B. Wang, Y. Zhang, Y. Shen, and Y. Kim. Parallelizing linear transformers with the delta

- rule over sequence length. *Advances in neural information processing systems*, 37:115491–115522, 2024.
- [73] S. Yang and Y. Zhang. Fla: A triton-based library for hardware-efficient implementations of linear attention mechanism, Jan. 2024.
- [74] C. Yu, X. Shu, Y. Wang, Y. Zhang, H. Wu, Y. Wu, R. Long, Z. Chen, Y. Xu, W. Su, and B. Zheng. Spiralformer: Looped transformers can learn hierarchical dependencies via multi-resolution recursion, 2026.
- [75] J. Yuan, H. Gao, D. Dai, J. Luo, L. Zhao, Z. Zhang, Z. Xie, Y. X. Wei, L. Wang, Z. Xiao, Y. Wang, C. Ruan, M. Zhang, W. Liang, and W. Zeng. Native sparse attention: Hardware-aligned and natively trainable sparse attention, 2025.
- [76] R.-J. Zhu, Z. Wang, K. Hua, T. Zhang, Z. Li, H. Que, B. Wei, Z. Wen, F. Yin, H. Xing, L. Li, J. Shi, K. Ma, S. Li, T. Kergan, A. Smith, X. Qu, M. Hui, B. Wu, Q. Min, H. Huang, X. Zhou, W. Ye, J. Liu, J. Yang, Y. Shi, C. Lin, E. Zhao, T. Cai, G. Zhang, W. Huang, Y. Bengio, and J. Eshraghian. Scaling latent reasoning via looped language models, 2025.

A. Adaptive Computation Time for Looped Transformers

This appendix gives a description of Adaptive Computation Time (ACT) for Looped Transformers and explains why we use a fixed number of loop iterations during pre-training, as stated in Section 2.

A.1. Per-Token, Input-Dependent Compute Allocation

In the basic Looped Transformer of Eq. (2), every token at every input is processed through exactly T loop iterations. ACT relaxes this constraint by letting the number of iterations depend on (i) the specific input sequence and (ii) the token position $t \in \{1, \dots, L\}$ within that sequence. We refer to this property as *adaptivity*: the model can spend more compute on token positions whose representations are still changing substantially between iterations and stop early on positions whose representations have already stabilized. Concretely, two tokens in the same sequence may halt at different iterations τ_t , and the same token may halt at different iterations across different inputs.

A.2. Halting Probabilities and Halting Rule

Following [19], ACT introduces a small auxiliary network—a single linear layer followed by a sigmoid in our implementation—that maps the hidden state at position t and iteration τ to a scalar in $(0, 1)$:

$$p_t^{(\tau)} = \sigma(\mathbf{w}^\top \mathbf{h}_t^{(\tau)} + b) \in (0, 1), \quad (8)$$

where $\mathbf{w} \in \mathbb{R}^d$ and $b \in \mathbb{R}$ are learned parameters shared across positions and iterations. We call $p_t^{(\tau)}$ the *halting probability*: it represents the model’s estimate, conditioned on the current hidden state, of how likely it is that no further loop iterations are needed for position t .

Iterations for position t accumulate until the running sum of halting probabilities crosses a threshold $1 - \epsilon$:

$$\tau_t = \min \left\{ \tau \mid \sum_{\tau'=1}^{\tau} p_t^{(\tau')} \geq 1 - \epsilon \right\}, \quad (9)$$

where ϵ is a small positive constant (we use $\epsilon = 0.01$, matching the value used by [16]). Once position t halts, its hidden state is frozen at $\mathbf{h}_t^{(\tau_t)}$ and does not participate in further updates. The threshold form $1 - \epsilon$, rather than exactly 1, ensures that the rule can be triggered after a single iteration if $p_t^{(1)}$ is sufficiently large; without the slack ϵ , at least two iterations would always be required because $p_t^{(\tau)} < 1$ by construction.

A.3. Pondering Cost and Turing Completeness

To prevent the model from trivially driving every $p_t^{(\tau)}$ to a small value and using the maximum allowed number of iterations on every token, ACT adds a *pondering cost* to the training loss that penalizes the expected number of iterations per position. The pondering cost is weighted by a scalar hyperparameter that controls the trade-off between accuracy and compute.

The combination of (i) input-dependent halting and (ii) unbounded effective depth is what allows looped Transformers with ACT to simulate arbitrary Turing machines, as shown by [49]. Without ACT, a Looped Transformer with a fixed number of iterations T has bounded computational depth and is therefore not Turing complete in the same formal sense.

A.4. Why We Use Fixed Iterations During Pre-training

Despite its theoretical appeal, ACT introduces several practical difficulties at pre-training scale:

- **Optimization instability.** The halting rule in Eq. (9) is non-differentiable, and the standard relaxation used by [16] couples the gradient of the pondering cost with the gradients of the main loss in ways that can produce sudden shifts in the average number of iterations during training.
- **Sensitivity to the pondering weight.** Small changes in the pondering hyperparameter can move the model between two degenerate regimes: halting after a single iteration on every token, or never halting until the maximum iteration cap is reached.
- **Throughput loss from ragged halting.** When different positions in the same batch halt at different iterations, the implementation must either pad to the longest unhalted position (losing the compute savings ACT was meant to provide) or use specialized ragged kernels.

Ouro [76] report these instabilities for the Ouro model family at pre-training scale, and our preliminary experiments reproduced the same behavior. We therefore use a fixed number of loop iterations T throughout pre-training in the main paper, and leave a stable ACT variant for future work.

B. Proofs for Section 2.2

B.1. Loop $\times$ DPLR linear attention: expressivity analysis

In this section, we analyze how unrolling a DPLR linear-attention block for T iterations enriches its state-transition operator. We show three things in sequence: (i) a single block applies a rank-1 update to the recurrent state, while T stacked blocks compose into an update of rank up to T ; (ii) by the Cartan–Dieudonné theorem, this composition is expressive enough to realize any orthogonal transformation in $O(d_k)$ once $T \geq d_k$; and (iii) as a concrete consequence, a single looped layer can compute prefix products for the symmetric group S_n whenever $T \geq n - 1$. Throughout, the spectral norm of the operator stays bounded by 1, so stability is preserved.

Setup. An LT₂-LA layer with a DPLR mixer maintains a recurrent state $\mathbf{S}_t \in \mathbb{R}^{d_k \times d_v}$ that evolves as

$$\mathbf{S}_t = \mathbf{A}_t \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top, \quad \mathbf{A}_t = \text{Diag}(\boldsymbol{\alpha}_t) (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top), \quad (10)$$

where the symbols denote the following quantities:

- $\mathbf{S}_t \in \mathbb{R}^{d_k \times d_v}$ is the recurrent state at step t , with key dimension d_k and value dimension d_v ;
- $\mathbf{k}_t \in \mathbb{R}^{d_k}$ is a unit-norm key vector ($\|\mathbf{k}_t\|_2 = 1$);
- $\mathbf{v}_t \in \mathbb{R}^{d_v}$ is the value vector;
- $\beta_t \in [0, 2]$ is a scalar gain controlling the strength of the rank-1 update;
- $\boldsymbol{\alpha}_t \in [0, 1]^{d_k}$ is a per-channel decay vector, applied as a diagonal matrix $\text{Diag}(\boldsymbol{\alpha}_t) \in \mathbb{R}^{d_k \times d_k}$;
- $\mathbf{I} \in \mathbb{R}^{d_k \times d_k}$ is the identity matrix.

Geometrically, the factor $(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top)$ is a generalized Householder transformation: it shrinks the component of any vector along $\mathbf{k}_t$ (by a factor $1 - \beta_t$) and leaves the orthogonal complement untouched. Multiplication by $\text{Diag}(\boldsymbol{\alpha}_t)$ then applies a per-channel decay. The whole map $\mathbf{A}_t \in \mathbb{R}^{d_k \times d_k}$ is therefore a *rank-1 perturbation of a diagonal matrix*.

When the same block is unrolled for T loops at step t with loop-indexed parameters $\{\boldsymbol{\alpha}_t^{(\tau)}, \beta_t^{(\tau)}, \mathbf{k}_t^{(\tau)}\}_{\tau=1}^T$, the cumulative state-transition operator becomes

$$\mathbf{A}_t^{\text{eff}} = \prod_{\tau=1}^T \mathbf{A}_t^{(\tau)} = \prod_{\tau=1}^T \text{Diag}(\boldsymbol{\alpha}_t^{(\tau)}) (\mathbf{I} - \beta_t^{(\tau)} \mathbf{k}_t^{(\tau)} \mathbf{k}_t^{(\tau)\top}) \in \mathbb{R}^{d_k \times d_k}. \quad (11)$$

From rank-1 updates to rank- T updates. A single $\mathbf{A}_t$ touches $\mathbf{S}_{t-1}$ along exactly one direction $\mathbf{k}_t$ (plus a channel decay), so it is a rank-1 correction to a diagonal map. Stacking T such factors performs T rank-1 corrections in succession; whether these corrections collapse or accumulate depends entirely on the geometry of the keys $\{\mathbf{k}_t^{(\tau)}\}_{\tau=1}^T$.

The two extremes make the picture clear (these special cases of generalized Householder products are well known; see e.g. [27]):

- *Identical keys* ($\mathbf{k}_t^{(1)} = \dots = \mathbf{k}_t^{(T)} = \mathbf{k}$). The product collapses: $\prod_{\tau=1}^T (\mathbf{I} - \beta_t^{(\tau)} \mathbf{k} \mathbf{k}^\top) = \mathbf{I} - \beta^* \mathbf{k} \mathbf{k}^\top$ for some scalar β^* . The rank stays at 1 and looping buys no expressivity.
- *Mutually orthogonal keys* ($\mathbf{k}_t^{(i)\top} \mathbf{k}_t^{(j)} = 0$ for $i \neq j$). The factors commute and combine cleanly into $\mathbf{I} - \sum_{\tau=1}^T \beta_t^{(\tau)} \mathbf{k}_t^{(\tau)} \mathbf{k}_t^{(\tau)\top}$, which is symmetric with rank exactly T in the perturbation term.

The take-away is that loop-induced rank is governed by the geometry of the keys across loop steps. This bodes well in practice: in high-dimensional spaces (d_k large), independently drawn unit vectors are nearly orthogonal with overwhelming probability. So for language modeling, where keys at different loop steps are generated from learned projections of the input, we should expect the keys to be approximately linearly independent and the effective rank of $\mathbf{A}_t^{\text{eff}}$ to be close to T rather than 1.

Reflections, rotations, and arbitrary orthogonal maps. Stepping up from rank to geometry, we now compare the transformations reachable by a single block versus T stacked blocks. With $\alpha_t^{(\tau)} = 1$ and $\beta_t^{(\tau)} = 2$, each factor $\mathbf{I} - 2\mathbf{k}_t^{(\tau)} \mathbf{k}_t^{(\tau)\top}$ is a Householder reflection. A single block can therefore realize any reflection, but cannot represent a rotation (rotations have determinant +1, reflections have determinant -1). Loop unrolling lifts this restriction:

Lemma 1 (Coordinate transpositions via reflection). *The permutation matrix $\mathbf{P}_{(i,j)} \in \{0, 1\}^{d_k \times d_k}$ that swaps coordinates i and j is realized by a single DPLR factor with $\alpha = 1$, $\beta = 2$, and $\mathbf{k} = \frac{1}{\sqrt{2}}(\mathbf{e}_i - \mathbf{e}_j)$, where $\mathbf{e}_i, \mathbf{e}_j \in \mathbb{R}^{d_k}$ are standard basis vectors.*

Proof. Direct expansion: $\mathbf{I} - 2\mathbf{k}\mathbf{k}^\top = \mathbf{I} - (\mathbf{e}_i - \mathbf{e}_j)(\mathbf{e}_i - \mathbf{e}_j)^\top$. Subtracting $(\mathbf{e}_i - \mathbf{e}_j)(\mathbf{e}_i - \mathbf{e}_j)^\top$ zeroes out the diagonal entries at (i, i) and (j, j) and places 1's at the off-diagonal entries (i, j) and (j, i) , exactly producing $\mathbf{P}_{(i,j)}$. $\square$

Theorem B.1 (Universal orthogonal representation). *Let $T \geq d_k$. For every orthogonal matrix $\mathbf{Q} \in \text{O}(d_k)$, there exists a configuration of per-loop parameters such that $\mathbf{A}_t^{\text{eff}} = \mathbf{Q}$.*

Proof. By the Cartan–Dieudonné theorem, every orthogonal matrix $\mathbf{Q} \in \text{O}(d_k)$ can be written as a product of at most d_k Householder reflections:

$$\mathbf{Q} = \prod_{\tau=1}^m (\mathbf{I} - 2\mathbf{k}^{(\tau)} \mathbf{k}^{(\tau)\top}), \quad m \leq d_k,$$

for some unit vectors $\mathbf{k}^{(\tau)} \in \mathbb{R}^{d_k}$. Set $\alpha_t^{(\tau)} = 1$, $\beta_t^{(\tau)} = 2$, and $\mathbf{k}_t^{(\tau)} = \mathbf{k}^{(\tau)}$ for $\tau \leq m$, and set $\beta_t^{(\tau)} = 0$ (identity factor) for $\tau > m$. Substituting into (11) gives $\mathbf{A}_t^{\text{eff}} = \mathbf{Q}$. $\square$

In particular, since rotations are products of an even number of reflections, two looped blocks suffice to realize any 2D rotation, four for any 3D rotation, and so on. This is the geometric content of the upgrade from rank-1 to rank- T : looping the same DPLR block converts a reflection-only operator into one that covers the full orthogonal group.

B.2. Loop $\times$ Sparse Attention: Receptive-Field Analysis

We analyse the receptive field of an LT2-SA layer that uses a static causal sparse attention pattern $\mathcal{M} = \{\mathcal{M}_i\}_{i=1}^N$, with $\mathcal{M}_i \subseteq \{1, \dots, i\}$ the keys visible to query i , looped for T iterations. Following Power attention and related discussion over receptive field [9, 67], we distinguish two notions:

- the *combinatorial* receptive field $\mathcal{I}_i^{(T)} \subseteq \{1, \dots, i\}$, defined as the set of input positions that *can* influence the loop- T output at i — equivalently, the set of nodes with a directed path to (i, T) in the layer-unrolled DAG induced by $\mathcal{M}$;
- the *effective* receptive field, defined as the set of input positions whose influence on the output is non-negligible once one accounts for softmax averaging and residual connections [42].

Section B.2.1 bounds $|\mathcal{I}_i^{(T)}|$ for an exhaustive list of static sparse patterns. Section B.2.2 then shows that residual connections can collapse linear or even exponential combinatorial reach to a constant effective horizon.

B.2.1. Combinatorial receptive field (no residual)

We treat the looped layer as a DAG: vertices (i, t) with $t \in \{0, \dots, T\}$, edges $(j, t-1) \rightarrow (i, t)$ iff $j \in \mathcal{M}_i$. Then $\mathcal{I}_i^{(T)} = \{j : (j, 0) \rightsquigarrow (i, T)\}$.

Sliding-window attention (SWA). For $\mathcal{M}_i = \{\max(1, i - w + 1), \dots, i\}$:

Proposition B.2 (Linear receptive-field growth under SWA looping). *For causal SWA with window w , looped T times,*

$$\mathcal{I}_i^{(T)} = \{\max(1, i - T(w - 1)), \dots, i\}, \quad |\mathcal{I}_i^{(T)}| = \min(i, T(w - 1) + 1) = \mathcal{O}(Tw).$$

Proof. By induction on T . For $T=1$, the claim restates the definition of a causal window of size w . Assume the claim for $T-1$. The loop- T state at position i is a function of the loop- $(T-1)$ states at positions $j \in \{\max(1, i - w + 1), \dots, i\}$, each of which by the inductive hypothesis depends on inputs in $\{\max(1, j - (T-1)(w-1)), \dots, j\}$. The union over j of these intervals equals $\{\max(1, i - T(w-1)), \dots, i\}$, completing the induction. $\square$

PowerAttention (power-of-two slashes). PowerAttention [9] couple SWA with $K = \lceil \log_2 N \rceil$ slash heads at strides $2^0, 2^1, \dots, 2^{K-1}$, so that each query directly attends to its local window plus the offsets $\{i - 2^k : 0 \leq k < K\}$. A single loop already reaches distance 2^{K-1} via the slashes, and the T -fold composition realises every distance expressible as a sum of T powers of two, giving

$$\mathcal{I}_i^{(T)} \supseteq \{i - d : 0 \leq d \leq \min(i - 1, 2^{T+K-1})\}, \quad |\mathcal{I}_i^{(T)}| = \mathcal{O}(\min(N, 2^T)).$$

This is the unique pattern in our list whose combinatorial receptive field grows *exponentially* in T [9, Thm. 3.2], while preserving the per-loop FLOP cost at $\mathcal{O}(Nw + N \log N)$, comparable to plain SWA.

B.2.2. Effective receptive field with residual connections

The bounds in Section B.2.1 are upper bounds on *topological* reach; they do not capture how much of an actual influence signal survives the cumulative softmax averaging and the residual short-circuit $h_t^{(\ell)} = h_t^{(\ell-1)} + \text{Attn}(h^{(\ell-1)})_t$ that every Transformer block — and hence every loop iteration of an LT2-SA layer — applies. We adopt a uniform-attention prior: each in-pattern key receives weight $1/|\mathcal{M}_i|$ on average. Let $P_T(d)$ denote the influence of the input at distance $d = i - j$ on the loop- T output at i .

No residual: Gaussian dilution and $\sqrt{T}$ growth. Without residuals, P_T is the T -fold convolution of the single-loop kernel P_1 . For SWA, P_1 is uniform on $[0, w - 1]$ with mean $\mu_1 = (w - 1)/2$ and variance $\sigma_1^2 = (w^2 - 1)/12$. By a CLT-type argument, repeated convolution drives P_T to a Gaussian:

$$P_T(d) \approx \mathcal{N}(d; T\mu_1, T\sigma_1^2), \quad D_{\text{eff}}^{\text{no-res}}(T) \approx 0.58 w \sqrt{T}.$$

Hence even though $|\mathcal{I}_i^{(T)}| = \Theta(Tw)$, the influence concentrates within an $\mathcal{O}(w\sqrt{T})$ band: information is diluted, and the effective receptive field grows only *sublinearly*.

With residual: a depth-independent exponential horizon. Modeling the residual as a convex mixture is a tractable proxy for the LayerNorm-induced effective contribution:

$$h_t^{(\ell)} = \alpha h_t^{(\ell-1)} + (1 - \alpha) \text{Attn}(h^{(\ell-1)})_t, \quad \alpha \in (0, 1),$$

with $\alpha \in [0.9, 0.99]$ typical at trained equilibrium, the influence kernel becomes a *spike-and-slab*: a mass of $\alpha + \frac{1-\alpha}{|\mathcal{M}_i|}$ at $d = 0$ and the residual share $1 - \alpha$ spread over $\mathcal{M}_i$. To travel a distance d exceeding the per-loop hop, information must take at least $\lceil d/w \rceil$ attention hops, each multiplying surviving mass by $(1 - \alpha)$, giving the exponential upper bound

$$P_T(d) \leq C (1 - \alpha)^{\lceil d/w \rceil},$$

which is asymptotically tight for $d \ll Tw$. Setting $P_T(D_{\text{eff}}) = \epsilon$ and solving yields a horizon *independent* of T :

$$D_{\text{eff}}^{\text{res}} \approx w \cdot \frac{\ln(1/\epsilon)}{\ln(1/(1 - \alpha))}. \quad (12)$$

For $\alpha = 0.95$ and $\epsilon = 10^{-2}$, this is $\approx 1.5 w$, regardless of how many times the layer is looped. Information from beyond ~ 2 – 3 window-widths is exponentially attenuated by the cumulative residual mass, and *additional loop iterations do not extend the effective horizon*.

C. Pre-training Setup

This appendix describes the experimental setup behind the language modeling results in Table 2. We pre-train every model from scratch on the FineWeb-Edu corpus [46] at two parameter scales, 0.6B and 1.3B, under a fixed 100B-token budget. The implementation lives in `apps/LT2` of our codebase and is built on the `lingua` framework [65]. All linear attention variants and NSA and from `fla` repo [73].

Data and tokenization. All runs draw from the FineWeb-Edu 100BT shard, packed at sequence length 4096. We tokenize with the Llama tiktoken tokenizer (vocabulary size 128,256) and prepend a BOS and append an EOS token to every document. The data loader runs asynchronously with a prefetch buffer of 1024 shards and produces two views per example for downstream consumption.

Token budget. Every model is trained for 255,000 optimizer steps at sequence length 4096. With the per-scale batch sizes given below, this corresponds to roughly 100 billion training tokens, i.e. a single epoch over the FineWeb-Edu 100BT shard. The same token budget is used for every variant in Table 2, so all comparisons are made at matched data.

Model scales. We use two parameter scales:

- **0.6B:** hidden dimension 1024, 25 physical layers, 16 attention heads, FFN multiple-of 256, RoPE base $\theta = 10,000$.
- **1.3B:** hidden dimension 2048, 16–25 physical layers (depending on the layer mix described below), 16 attention heads, same FFN and RoPE settings.

We refer to the larger scale as “1.3B” throughout because the exact parameter count varies slightly across variants (e.g. DSA layers are heavier than dense Transformer blocks), but width and head count are held fixed.

Looped variants. Every looped variant uses $T = 4$ loops over the physical layer stack: the same parameters are applied four times in sequence, so the effective depth is $4 \times n_{\text{layers}}$ while the parameter count stays at the single-pass value. We add learned residual scaling between iterations (`use_residual=true`) and do not use cross-block residuals. Non-looped baselines run a standard Transformer stack of the same physical depth.

Layer mixes. The “hybrid” variants in Table 2 interleave a sub-quadratic mixer with periodic full softmax-attention layers in a 4:1 pattern: four mixer layers followed by one full-attention layer, repeated. Concretely, for the 1.3B hybrid runs the physical stack has 16 mixer layers and 4 (or 3) full-attention layers; for 0.6B the stack has 17 mixer layers and 4 full-attention layers. The mixer family varies by row and includes Gated DeltaNet (GDN), DeltaNet, KDA, NSA, RetNet, HGRN2, Mamba2, MLA, and DSA. Pure baselines replace the mixer slots with the same operator and remove the full-attention layers. Sub-quadratic mixers use the `flash-linear-attention` kernels; full-attention layers use FlashAttention-3, except for sliding-window variants which use the FMHA path with `window_left = w-1`, `window_right = 0` and a default window size of $w = 256$ at 0.6B and $w = 2048$ at 1.3B.

Optimization. We use AdamW with $(\beta_1, \beta_2) = (0.9, 0.95)$, gradient clipping at norm 1.0, and bf16 mixed-precision training. Peak learning rate is 3×10^{-4} for 1.3B runs and 1.5×10^{-4} for 0.6B runs (lowered for stability with the looped sliding-window 0.6B model). The schedule is linear warmup followed by cosine decay to a floor of $10^{-6} \times$ the peak rate. Warmup is 5,000 or 10,000 steps depending on the variant. Weight decay is 0.1 for almost all runs (0.033 for the 1.3B 4:1 window baseline, where the smaller value was needed to keep training stable). All runs use seed 777 for the trainer and seed 42 for model initialization.

Distributed setup. Training is distributed with FSDP (`full_shard`) and `torch.compile` enabled wherever the kernel allows it (we disable compilation only for DSA, whose TileLang kernel is incompatible with `torch.compile`). We do not use tensor parallelism. Per-GPU micro-batches of 2–12 sequences are combined with gradient accumulation between 1 and 6 steps to reach a global token-per-step target of roughly 4×10^5 tokens; combined with 255,000 steps this gives the $\sim 100\text{B}$ -token budget.

FLOP accounting. For looped models we count FLOPs over the unrolled depth, i.e. we multiply the per-layer FLOP count by $n_{\text{layers}} \times T$ rather than by n_{layers} alone. This means the FLOP-per-token figures we report for looped variants reflect the actual compute spent during training, not the unique parameter count.

Evaluation. We report validation loss/perplexity on a held-out FineWeb-Edu 10BT validation shard and zero-shot accuracy on the LM-Evaluation-Harness suite: HellaSwag, BoolQ, PIQA, SocialIQA, WinoGrande,

OpenBookQA, ARC-Easy, ARC-Challenge, RACE, CommonsenseQA, and COPA. Numbers in Table 2 use the final checkpoint of each run.